Harness

AI-powered coding agent using DeepSeek.

中文文档

Contents

    - History Compaction
    - Tool Result Distillation

Setup

```
# Install dependencies
npm run install:all
```

Configuration

Harness uses a **client-entered API key** model. Enter your DeepSeek API key once in the Harness UI under the model selector:

1. Click the model selector in the agent console
2. Enter your API key (starts with `sk-...`)
3. Click Save

The key is sent once to the server and stored persistently on disk (`~/.harness/store/api-keys.enc`, AES-256-GCM encrypted) — it is never persisted in browser `localStorage`, survives app restarts and updates, and is never re-sent in agent request bodies. The key remains stored until explicitly removed via "Remove API Key" in the UI or the `~/.harness/` directory is deleted.

All client-side state (selected model, chat history, recent folder paths, open editor tabs, model presets, terminal history) is stored in browser `localStorage` and **mirrored to** `~/.harness/store/client-state.json` on every change and on app exit. This ensures data survives reinstalls, since `%USERPROFILE%\.harness\` is outside the Electron installer's scope. On startup, the client fetches `GET /api/client/state` and restores any previously saved state.

Get a key at platform.deepseek.com.

Start

```
npm run dev
```

This starts both the backend and frontend. The OS assigns both ports; check console output for the URLs.

Port discovery flow:

1. Express starts → `server.listen(0)` → OS assigns a free port → port written to `~/.harness/ports/express-port`

2. Vite starts immediately (no blocking) → proxies `/api`, `/ws`, `/_browser` via middleware that reads `~/.harness/ports/express-port` on each request → returns `503 Service Unavailable` until Express is live, then forwards normally
3. Vite binds → `port: 0` → OS assigns a free port → port written to `~/.harness/ports/vite-port`
4. Electron (desktop mode) reads both files to connect to Express and load the Vite dev page

Both servers let the OS decide — no hardcoded port numbers anywhere.

Stale port cleanup: On shutdown (Ctrl+C, SIGTERM), Express deletes the port file. If Express crashes unexpectedly and the file lingers, the Vite proxy middleware detects the dead port (`ECONNREFUSED`), invalidates the cached port, and re-reads the file on the next request. Port files are stored in `%TEMP%/harness-ports/` (platform temp directory) to avoid filesystem permission issues on sandboxed environments.

Single-instance lock: The desktop app uses a custom PID-file lock instead of Electron's `app.requestSingleInstanceLock()` (which is unreliable on Windows sandboxed environments). On startup, it writes the current PID to `%TEMP%/harness-pid`; if a PID file already exists with a live process, the new instance quits. On clean shutdown, the PID file is removed. This prevents port file trampling and shared-state (`~/.harness/` files) corruption that would occur if two Express servers competed for the same resources.

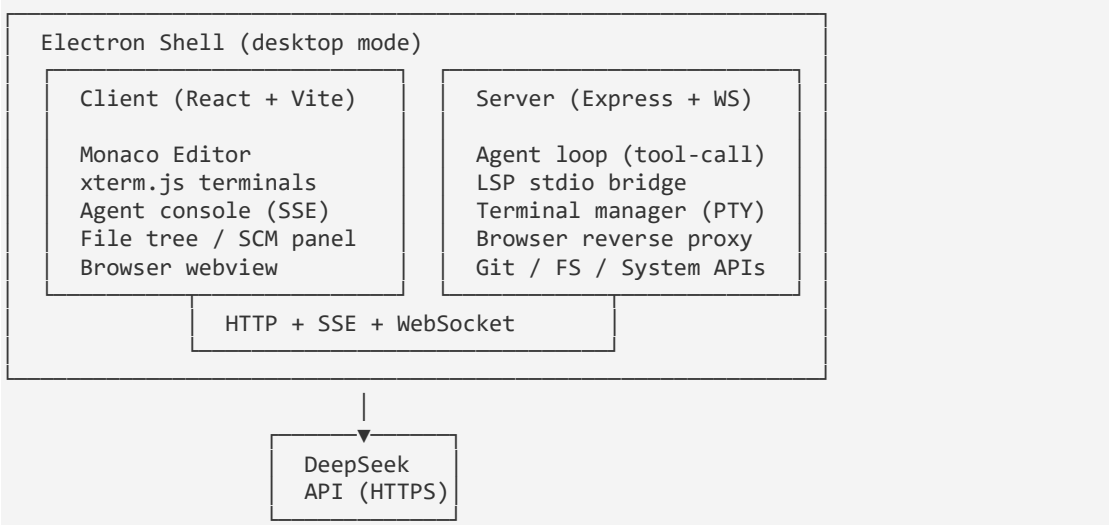
File integrity: All `~/.harness/` files are local to the user's machine. If modified by external actors, the effects are non-destructive — the app detects corruption and resets gracefully:

File	If tampered
`ports/express-port`	`waitForOwnServerPort` validates via `/api/health` + PID check. Wrong PID → timeout → app shows startup error.
`ports/vite-port`	Electron loads wrong URL → connection refused → loading screen with timeout.
`store/client-state.json`	`JSON.parse` failure → all state resets to defaults. Valid but wrong JSON → UI shows bad model/paths; model strings just fail API calls; paths are displayed, never auto-opened.
`store/api-keys.enc`	AES-256-GCM auth tag mismatch on decrypt → API keys reset. File is unreadable without the machine key at `~/.harness/.key`.
`store/memory.db`	SQLite corruption → memory features reset.

File	If tampered
`.key`	Replaced or deleted → existing `api-keys.enc` becomes permanently unreadable (new key generated on next save).

Architecture

Harness is a client-server application with an optional Electron desktop shell.



Server (`server/`)

The Node.js Express server is the backbone. It owns all backend logic and never runs in the browser. The OS assigns a free port at startup, written to `~/.harness/ports/express-port` for discovery.

Layer	File	Role
HTTP API	`server/index.ts`	REST endpoints for filesystem CRUD, git status/commit/diff, project detection, system stats, and agent chat (blocking + SSE streaming + step-by-step)
Agent loop	`server/agent.ts`	Tool-calling orchestration: receives user messages, sends tool definitions to DeepSeek, executes filesystem/terminal tools, manages browser tool handoff, compacts conversation history

Layer	File	Role
DeepSeek bridge	`server/deepseek.ts`	Raw DeepSeek API calls — chat, tool-calling, and SSE streaming — with prefix-cache tracking plus API-backed usage and cache-token reporting
LSP bridge	`server/lsp.ts`	Spawns language servers over stdio, forwards diagnostics to the client, handles completions and hover
Terminal manager	`server/terminalManager.ts`	Creates per-session shell processes (PTY via `node-pty` or pipe fallback), routes I/O between client WebSocket messages and child process stdio, auto-detects localhost URLs in terminal output
Browser proxy	`server/index.ts` (`/_browser`)	Reverse-proxies external URLs through the server so the client iframe stays same-origin, strips `X-Frame-Options` headers, injects a restrictive CSP

3 transport channels to the client:

- **HTTP** — Standard REST for file reads/writes, git operations, LSP diagnostics, agent chat init
- **SSE (Server-Sent Events)** — One-way streaming for agent thinking/text/tool events during an agent turn
- **WebSocket** — Bidirectional for terminal I/O (`term:create`, `term:write`, `term:resize`, `term:kill`) and server-to-client broadcasts (logs, errors, browser URL detection)

Client (`client/`)

The React + Vite frontend runs on an OS-assigned port (`port: 0`) in development. In desktop/production mode, the Express server serves the built static files directly from `client/dist/`.

Pane	File	Role
Editor	`EditorPane.tsx`	Monaco Editor with tabs, file tree, SCM panel, built-in browser webview, and terminal tabs —

Pane	File	Role
		the main workspace
Agent console	`AgentConsole.tsx`	Chat interface for the AI agent. Sends user goals to `/api/chat/agent/stream`, consumes the SSE event stream, renders tool calls with spinners/results, prompts user for permission on `run_in_terminal`, and shows Accept/Reject diffs for file edits
Files panel	`FilesPanel.tsx`	File explorer tree with create/rename/delete, right-click context menu, and folder expansion state persistence
Terminal	`TerminalPane.tsx`	xterm.js terminal tabs, connected via WebSocket to the server's terminal manager
SCM panel	`ScmPanel.tsx`	Git staging area, commit history, diff viewer
Status bar	`StatusBar.tsx`	Language selector, encoding, indentation, cursor position, go-to-line
Menu bar	`MenuBar.tsx`	File/Edit/View/Terminal/Help menus

The client **never calls DeepSeek directly**. All AI interaction flows through the server's agent loop, which owns the API key and tool execution.

Data flow (agent turn)

```
User types goal → AgentConsole
  → POST /api/chat/agent/stream (message, context, projectRoot)
     or POST /api/chat/agent/stream/stepbystep (IDE-driven mode)
  → Server builds dynamic system prompt (ITR), compacts history, calls DeepSeek
  → DeepSeek returns text/tool_calls via SSE stream
  → Server executes filesystem tools (read_file, write_file, etc.) directly
  → Interactive browser tools (click, type, etc.) yield SSE "browser_tool" event
(sub-agent only)
  → AgentConsole sends browser command to EditorPane's webview
  → WebView executes the action, returns result
  → AgentConsole calls POST /api/chat/agent/stream/continue (toolCallId, result)
  → Loop continues until write_summary + task_complete
  → Successful write_summary is shown once in agent-body as markdown preview
```

Agent loop internals

The agent loop (`agentLoopStream` / `agentLoop` in `server/agent.ts`) orchestrates a conversation between the user, the model, and tools using a message array (`state.messages`). Each turn follows a fixed pattern:

```
┌──────────────────────────────────────────────────────────────────┐
│  Agent loop iteration                                            │
│                                                                  │
│  1. buildOpenAiMessages(state)                                   │
│     ↓ Converts internal messages → DeepSeek API format           │
│     ↓ Pairs tool_calls with tool_call_id responses               │
│     ↓ Injects system prompt (ITR-selected chunks)                │
│                                                                  │
│  2. chatDeepSeekToolStream(messages, tools)                      │
│     ↓ Sends to DeepSeek, receives SSE stream                     │
│     ↓ Yields thinking events, text, tool_calls                   │
│                                                                  │
│  3. For each tool call:                                          │
│     ├─ Push assistant tool_calls message to state.messages       │
│     ├─ Execute tool (filesystem / terminal / browser)            │
│     ├─ Push tool result message to state.messages                │
│     └─ Continue to next tool (batch) or next iteration           │
│                                                                  │
│  4. If no tool calls → final text reply (only when no work was performed),
otherwise must write_summary + task_complete │
│     → write_summary stores the final summary, AgentConsole renders it once in
`agent-body`, then `task_complete` returns phase: "done" │
└──────────────────────────────────────────────────────────────────┘
```

Message roles

The agent state tracks three message roles. Each serves a specific purpose in the conversation:

Role	Who creates it	Purpose	Content
`user`	Client (`createAgentSession`)	User's request / goal	Plain text ("Add login endpoint")
`assistant` (text)	DeepSeek API → server pushes	Model's reasoning / replies	Text response
`assistant` (with `name`)	DeepSeek API → server pushes	Tool call request	JSON array of `{ id, function: { name, arguments } }`
`tool`	Server (after tool execution)	Tool execution result	Tool output (file contents, command output, browser result)

Message lifecycle

```
User sends request
  → state.messages = [{ role: "user", content: "..." }]

Iteration 1: DeepSeek decides to read a file
  → state.messages.push({ role: "assistant", name: "read_file", content: '[...]' })
  → Server executes read_file → returns file contents
  → state.messages.push({ role: "tool", content: "...", tool_call_id: "call_1" })
```

```
Iteration 2: DeepSeek reads the result, decides to edit
  → state.messages.push({ role: "assistant", name: "edit_file", content: '[...]' })
  → state.messages.push({ role: "tool", content: "Wrote ...", tool_call_id:
"call_2" })

Iteration 3: DeepSeek is done
  → Calls write_summary → stores final structured summary
  → AgentConsole renders that summary once as an assistant message in `agent-body`
    using markdown preview (`###` headings, lists, inline code)
  → Calls task_complete → agent returns phase: "done" using the stored summary
```

Each tool call is always a matched pair: an `assistant` message with `name` containing the tool_calls JSON, followed immediately by a `tool` message with the same `tool_call_id`. `buildOpenAiMessages()` enforces this pairing — unpaired tool calls get synthetic error responses before the request reaches DeepSeek.

Desktop vs web mode

Feature	Web (browser)	Desktop (Electron)
Server	External process (`npm run dev:server`)	Embedded via `tsx` require in the Electron main process
Client	Vite dev server (OS-assigned port) or served by Express (production)	Vite dev server in dev, served by Express in production
Terminal	WebSocket to server, pipe-fallback PTY	WebSocket to server, `node-pty` with ConPTY on Windows
File access	Browser File System Access API or server FS APIs	Server FS APIs + native Electron `dialog` for folder/file pickers
Built-in browser	iframe + reverse proxy (`/_browser`)	Electron `webview` with geolocation, permissions, popup interception

Desktop (Electron)

Harness can also run as a desktop app (closer to VS Code) with an embedded server and a PTY-backed terminal.

```
# Desktop dev (runs Vite + Electron)
npm run desktop:dev
# Build the client for desktop packaging
npm run desktop:build

# Package a Windows build (electron-builder)
npm run desktop:pack
```

Notes:

- `npm install` runs `electron-rebuild` for `node-pty` automatically (via `postinstall`).
- The terminal prefers `node-pty` (ConPTY on Windows) and falls back to pipe mode if PTY isn't available.

Built-in Browser (Desktop)

In Electron mode, Harness includes a full browser in the editor area powered by an Electron `webview`.

Features

Auto-detect localhost URLs — When a terminal process outputs a URL, Harness scans the output in real time and opens compatible URLs in a new browser tab automatically.

- **Detection pattern:** Any URL matching `http(s)://localhost`, `127.0.0.1`, `0.0.0.0`, or `[::1]` with a port number (e.g. `http://localhost:5173`).
- **Web page vs API filtering:** Before opening, Harness sends a quick `HEAD` request to check the `Content-Type` header. Only URLs returning `text/html` are opened as browser tabs — API endpoints (e.g. `/api/health`, JSON responses) are silently skipped.
- **Deduplication:** Each URL is opened at most once per terminal session. Repeating the same URL in terminal output is ignored.
- **Supported sources:** Works with both PTY and pipe-based terminals, scanning both stdout and stderr.

Manual navigation — Type a URL or a Bing search query in the address bar and press Enter or click Go.

Back / Forward / Refresh — Toolbar buttons with disabled state when navigation isn't available.

Site information — Click the security icon to see the connection status (secure/not secure), the current URL, and permission toggles.

Site permissions — Per-origin toggles for:

- **Geolocation** — Uses Windows native location via PowerShell `GeoCoordinateWatcher` (no Google API key required). Location is cached IDE-wide and refreshed every 5 minutes. Works across all navigation without re-granting.
- Camera / Microphone / MIDI / Autoplay

Tabbed browsing — Multiple browser tabs can be open at the same time, just like file tabs.

Title syncing — The browser tab label follows the page's `<title>`.

Pop-up interception — Links that would open a new Electron window are captured and opened as a new Harness browser tab instead.

Cross-navigation location — `navigator.geolocation` is overridden at `dom-ready` so the page always uses Harness's native Windows location bridge, even after navigating between routes.

> Note: Geolocation requires `https://` or `localhost`. The Windows Location API must be enabled in Windows Settings (`Privacy > Location`).

Language Support (LSP)

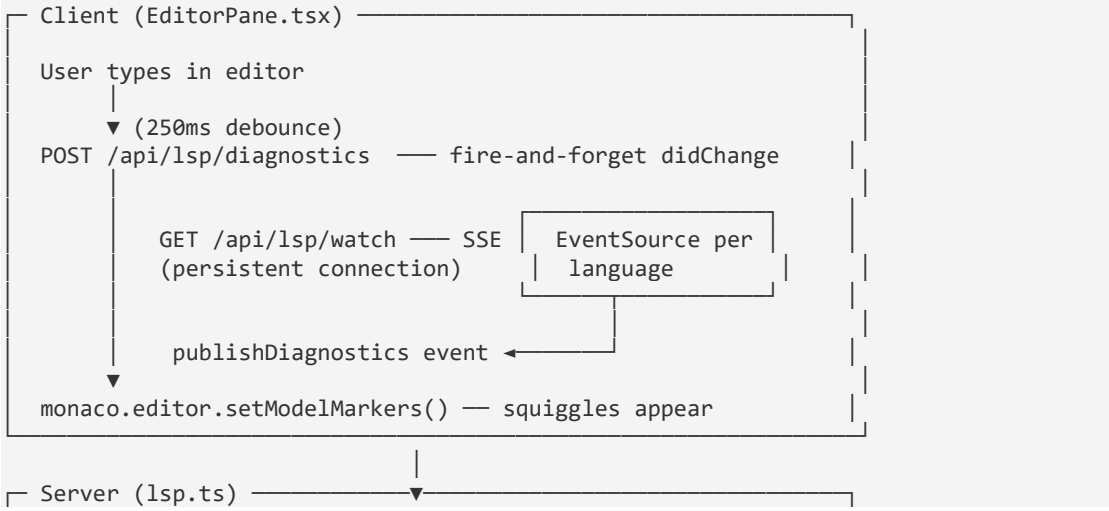
Harness provides editor intelligence — continuous error/warning checking, completions, and hover — through two layers:

1. Built-in (no setup): Monaco validates these in the browser, live as you type:

- JavaScript / TypeScript (JSX/TSX)
- JSON, CSS / SCSS / LESS, HTML

2. Language Server (LSP): For everything else, Harness talks to a standard language server over stdio (`server/lsp.ts`). The architecture follows the VS Code model — push-based, real-time diagnostics via Server-Sent Events (SSE).

Architecture (VS Code-style push model)



Key differences from polling-based approaches:

Aspect	Old (polling)	New (VS Code-style)
Diagnostics delivery	Client polls `/api/lsp/diagnostics` every 250ms	LSP server pushes via SSE — instant
Cross-file analysis	Only the changed file was polled	All files receive diagnostics from any change
URI handling	Polling used module-level Map with manual key normalization	SSE streams normalized URIs directly to matching file
Connection	One HTTP request per file change	One persistent SSE connection per language

How it works:

1.

SSE connection — When a file is opened, the client establishes a persistent `GET /api/lsp/watch?rootPath=...&language=...` SSE connection per language. The server holds the connection open and registers it in `session.sseClients`.

2.
3.

File change notification — On content change (250ms debounce), the client fires a `POST /api/lsp/diagnostics` with the file text. The server sends `textDocument/didOpen` or `textDocument/didChange` to the LSP process and returns immediately (fire-and-forget).

4.
5.

Diagnostics push — When the LSP server emits `textDocument/publishDiagnostics`, `handleMessage()` broadcasts the markers to ALL connected SSE clients for that language. The client receives the event, matches the URI to an open file, and calls `monaco.editor.setModelMarkers()` to render squiggles.

6.
7.

Cross-file analysis — Because pyright (and other LSP servers) scan the entire workspace on any change, diagnostics for ALL files arrive via SSE and are applied simultaneously. Opening file2 immediately shows errors that pyright published during file1's analysis.

8.

A language server is only used **if its executable is found on your** `PATH`. If it isn't installed, that language is simply skipped — no errors, no setup required.

URI normalization

Different LSP servers encode file URIs differently — pyright uses `%3A` for drive letters and `%5C` for backslashes, pylsp uses bare characters, some double-encode. The `normalizeUri()` function in `server/lsp.ts` handles all variants:

- Progressive `decodeURIComponent` (handles double-encoding like `%2520`)
- Per-character fallback for mixed raw/encoded URIs
- Backslash → forward slash normalization
- Case-insensitive matching (lowercase)

Supported languages and their servers

Language	Server binary	Install (example)
Python	`pyright-langserver` (preferred) `pylsp` (fallback)	`npm i -g pyright` `pip install python-lsp-server pyflakes`
JavaScript / TS	*(Monaco built-in)*	—
HTML / CSS / JSON	*(Monaco built-in)*	—
Java	`jdtls`	install Eclipse JDT Language Server

Language	Server binary	Install (example)
C#	`omnisharp`	install OmniSharp (`-lsp`)
C / C++	`clangd`	install LLVM/clangd
Go	`gopls`	`go install golang.org/x/tools/gopls@latest`
Rust	`rust-analyzer`	`rustup component add rust-analyzer`
Ruby	`solargraph`	`gem install solargraph`
PHP	`intelephense`	`npm i -g intelephense`
Swift	`sourcekit-lsp`	ships with the Swift toolchain
Kotlin	`kotlin-language-server`	install kotlin-language-server
Markdown	`marksman`	install marksman
YAML	`yaml-language-server`	`npm i -g yaml-language-server`
SQL	`sqls`	`go install github.com/lighttiger2505/sqls@latest`

> Additional servers are also mapped out of the box (Lua, Dockerfile, Vue, Svelte, Dart, Elixir, Haskell, Terraform, Clojure, OCaml, Zig, Scala, TOML, Bash) — install the corresponding binary and reload.

After installing a server, **restart the backend** (`npm run dev:server`, or `npm run dev`) so the new executable is detected.

Reducing false positives

Harness applies multiple layers of filtering to keep diagnostics high-signal:

Server-side diagnostic pipeline (`lsp.ts` — `handleMessage`):

- **Severity filtering** — Info (3) and Hint (4) diagnostics are dropped. Only Error and Warning reach the editor.
- **Validity checks** — Diagnostics with negative line/column positions or inverted ranges (end before start) are discarded.
- **Per-file cap** — Max 200 diagnostics per file to prevent UI flooding on legacy or untyped code.

Per-language LSP server tuning (applied via `workspace/didChangeConfiguration` at init):

Language / Server	Tuning
JavaScript (Monaco built-in)	Syntax-only validation; semantic/type checks disabled (no module graph in plain JS)
TypeScript (Monaco built-in)	Full semantic checking with `strict: false`, `noImplicitAny: false`
Python / pyright	`typeCheckingMode: "basic"`, `diagnosticMode: "openFilesOnly"`; `reportOptional*` rules → `"none"` (idiomatic Python uses Optional without guards); `reportMissingImports` → `"warning"` (venv/monorepo resolution gaps); `reportAttributeAccessIssue`, `reportArgumentType`, `reportAssignmentType` → `"warning"`
Python / pyright (venv)	Auto-detects `.venv` / `venv` / `env` / `.env` under the project root and passes `venvPath` + `venv` so pyright resolves site-packages
Python / pylsp	Keeps `pyflakes` (real bugs); disables `pycodestyle`, `pydocstyle`, `mccabe`, `flake8`, `pylint`
YAML	Schema-store lookups disabled — avoids "Schema not found" false positives when no matching schema exists
Go / gopls	`staticcheck: false` — disables opinionated style suggestions

Engine-level fixes:

- `mapSeverity` defaults undefined severity to **Error** (LSP spec: omitted means error), not Warning.
- Diagnostics cache is scoped per project root — SSE init flushes only the current session's URIs, not the global map.
- Diagnostic entries are cleaned from the cache when the LSP session exits, preventing stale markers.

Adding a language

Add an entry to `SERVER_SPECS` in `server/lsp.ts` mapping the language id to its server binary, and (if needed) the file extension in `detectLanguage` in `client/src/panes/fileModel.ts`.

File Management

Harness includes a file explorer tree (`FilesPanel`) with full create, delete, and rename capabilities — both for your manual use and for the AI agent.

Creating files

- Click the **+** button in the files header to create a new file. If no folder is selected in the tree, the file is created in the project root. If a folder is **selected** (click it once — it highlights), the new file is created inside that folder.
- Folders are automatically created on demand when you add a file under a path that doesn't exist yet.

Right-click context menu

- Right-click any item in the file tree to **Rename** or **Delete** it.
- Deleting a folder removes it recursively.

AI Agent file access The AI agent has filesystem tools:

Tool	Description
`read_file`	Reads a file with line numbers (or lists a directory)
`write_file`	Creates or overwrites a file with full content
`edit_file`	Targeted string replacement — send only the lines that change
`list_files`	Lists directory contents (skips `.git` / `node_modules`)
`search_files`	Recursively find files/folders by name pattern
`grep`	Search file contents for a regex pattern
`create_directory`	Creates a new directory (and any parent dirs)
`rename_file`	Renames or moves a file or directory
`delete_file`	Deletes a file or directory (recursively)

All tools operate relative to the project root. The agent can browse, create, edit, and clean up files on its own — no manual intervention needed.

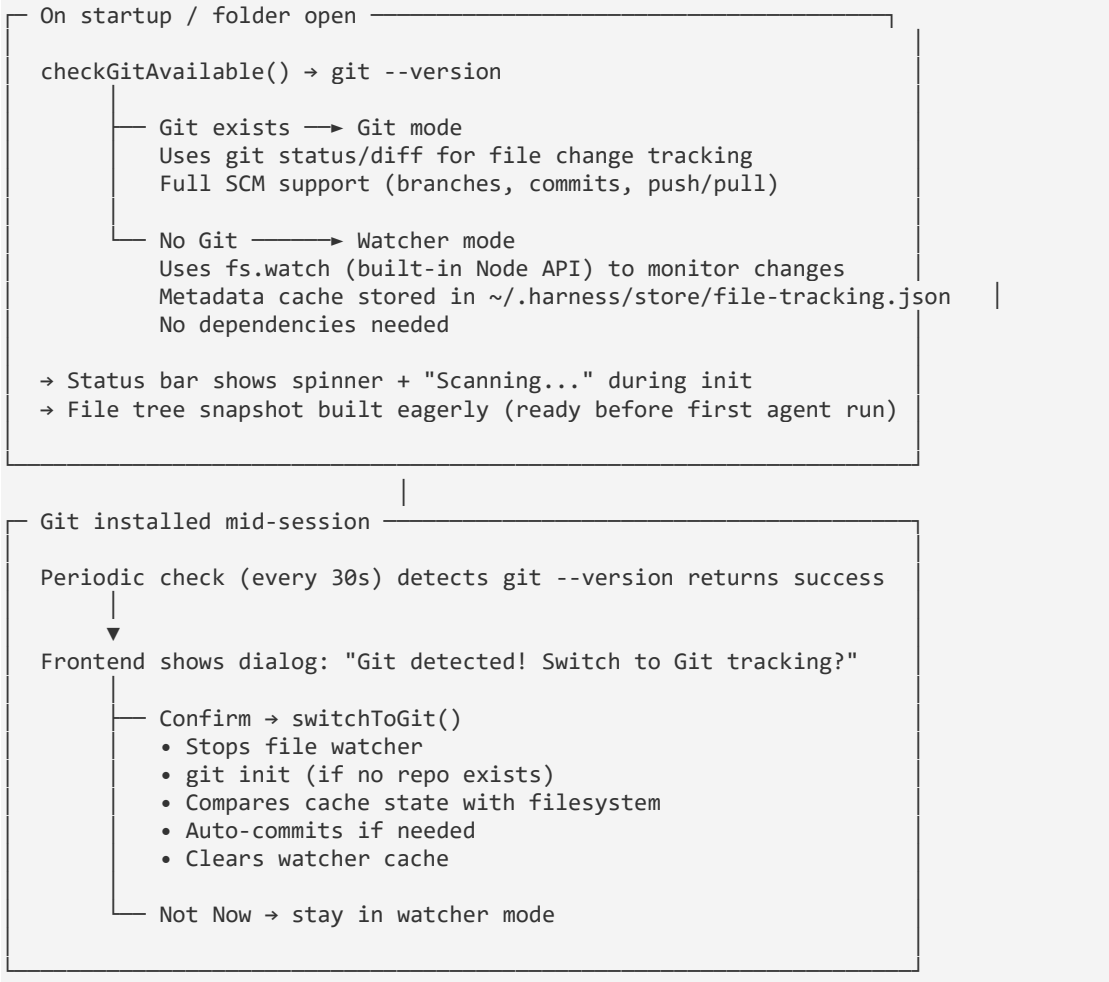
Server APIs

Endpoint	Method	Description
`/api/fs/create-file`	POST	Creates a file (and parent dirs) if it doesn't already exist
`/api/fs/delete`	DELETE	Deletes a file or directory (recursive for dirs)
`/api/fs/rename`	POST	Renames / moves a file or directory
`/api/fs/read-binary`	GET	Read a file as binary (used by `browser_upload_file`)

Smart File Tracking

Harness uses a dynamic file tracking system that auto-detects Git availability — following the "workspace trust" pattern used by modern IDEs.

How It Works



Tracking Modes

Mode	Detection	File Changes	SCM Panel	Status Bar
git	Git found on startup or switch	`git status --porcelain`	Fully functional	Branch icon + "main"
watcher	No Git available	`fs.watch` recursive + JSON cache	Disabled	Crosshair icon + "Watcher"
loading	Folder just opened	Scanning filesystem + building snapshot	—	Spinner + "Scanning..."
none	No folder open	—	—	—

File Tree Context for AI Agent

When the agent runs, Harness sends the project file tree as part of the system prompt context. The snapshot is built **eagerly** on folder open (not deferred to the first agent call), so it's always ready.

```
┌─ Folder open ──────────────────────────────────────────────────────┐
│  → buildSnapshot() walks entire project (skips node_modules/.git)  │
│  → Knowledge graph built (~/.harness/snapshots/file-tree-snapshot-<hash>.kg) │
│  → Visualization written to ~/.harness/snapshots/file-tree-snapshot-<hash>.txt │
│  → Status bar: spinner + "Scanning..." during the walk             │
└────────────────────────────────────────────────────────────────────┘
                              │
┌─ First agent run ──────────────────────────────────────────────────┐
│  → "(no file tree changes since last update)" — snapshot matches   │
└────────────────────────────────────────────────────────────────────┘
                              │
┌─ Subsequent agent runs (same folder) ──────────────────────────────┐
│  → Only patches sent: "+ added files" / "- deleted files"          │
│  → Snapshot updated after each send                                │
└────────────────────────────────────────────────────────────────────┘
                              │
┌─ Large changes (>100 files differ) ────────────────────────────────┐
│  → e.g. after git checkout to a different branch                   │
│  → Falls back to sending a full tree instead of a massive patch    │
│  → Snapshot updated, subsequent calls return to normal patch mode  │
└────────────────────────────────────────────────────────────────────┘
                              │
┌─ Cross-session continuity ─────────────────────────────────────────┐
│  → Snapshot persists to disk                                       │
│  → Restarting IDE does not re-send full tree unless folder changed │
│  → New folder open → snapshot rebuilt → ready before first run     │
└────────────────────────────────────────────────────────────────────┘
```

Workspace Deduplication

Each workspace gets a unique snapshot filename keyed by an MD5 hash of its resolved absolute path. This prevents cross-project collisions and ensures the same folder always maps to the same graph file.

```
d:\Work Projects\Harness   → MD5 → a1b2c3d4e5f6
                             → ~/.harness/snapshots/file-tree-snapshot-
a1b2c3d4e5f6.kg
                             → ~/.harness/snapshots/file-tree-snapshot-
a1b2c3d4e5f6.txt

d:\Other Projects\app       → MD5 → f6e5d4c3b2a1
                             → ~/.harness/snapshots/file-tree-snapshot-
f6e5d4c3b2a1.kg
                             → ~/.harness/snapshots/file-tree-snapshot-
f6e5d4c3b2a1.txt
```

- **Same folder, same hash** — reopening a project overwrites its existing snapshot (no stale duplicates).
- **Different folders, different hashes** — each workspace has independent graph files.
- **Path changes break the link** — renaming or moving the project folder produces a new hash and a fresh snapshot. The old file is orphaned (not auto-cleaned).
- `read_graph` **uses identical hashing** — the tool locates the correct `.kg` file at query time by computing the same MD5 from the project root.

API Endpoints

Endpoint	Method	Description
`/api/file-tracking/status`	GET	Current tracking mode, Git availability, workspace path
`/api/file-tracking/init`	POST	Initialize tracking for a workspace (`{ workspacePath }`)
`/api/file-tracking/git-detected`	GET	Polled by frontend — returns `{ gitDetected: true }` when Git appears mid-session
`/api/file-tracking/switch-to-git`	POST	Switch from watcher to git; auto-inits repo, compares state
`/api/file-tracking/changes`	GET	Get changed files (works in both modes)
`/api/file-tracking/refresh`	POST	Force re-scan workspace in watcher mode
`/api/file-tracking/file-tree-`	GET	Full tree (first call) or patch (subsequent calls) for agent system prompt

Endpoint	Method	Description
`context`		
`/api/file-tracking/reset-snapshot`	POST	Reset snapshot so next context call returns full tree

Files

File	Role
`server/fileTracking.ts`	`FileTrackingService` — singleton orchestrating Git or watcher mode, periodic Git detection, snapshot/patch logic
`server/fileTrackingStore.ts`	`FileTrackingStore` — lightweight JSON-backed cache (`~/.harness/store/file-tracking.json`) for file metadata
`server/knowledgeGraph.ts`	`buildKnowledgeGraph()` — builds codebase graph with CONTAINS + IMPORTS edges, `.kg` serialization, `.txt` visualization
`~/.harness/store/file-tracking.json`	On-disk metadata cache for watcher mode
`~/.harness/snapshots/file-tree-snapshot-<hash>.kg`	Per-workspace Knowledge Graph — nodes (dirs/files) + CONTAINS edges + parsed IMPORTS
`~/.harness/snapshots/file-tree-snapshot-<hash>.txt`	Human-readable visualization sidecar (nested tree with import annotations)

Knowledge Graph

Harness builds a **codebase knowledge graph** on folder open — a structured representation of every file, directory, exported symbol, and their relationships. This feeds the agent's system prompt as a compact nested tree and persists to disk as a `.kg` file for graph-based queries.

Schema

The graph has three node types and four edge types:

Node Type	Field	Description
`dir`	`name`	Directory
`file`	`name`, `kind` (extension)	Source file, config, doc
`symbol`	`name`, `kind`	Exported function, class, const, type, interface, enum, or default export

Edge Type	From → To	Description
`CONTAINS`	dir → file | dir	Structural: parent directory contains child
`EXPORTS`	file → symbol	A file exports a named symbol
`IMPORTS`	file → file	File-level import (e.g. `import './utils'`)
`IMPORTS_SYMBOL`	file → symbol	Precise symbol-level import (e.g. `import { foo } from './utils'`)

Symbol Parsing

For TypeScript/JavaScript files (`.ts`, `.tsx`, `.mts`, `.cts`), the TypeScript compiler API parses the AST to extract exports:

Kind	Detected from
`function`	`export function foo()`
`class`	`export class Foo {}`
`const`	`export const x = ...`, `export { x }`
`type`	`export type T = ...`
`interface`	`export interface I {}`
`enum`	`export enum E {}`
`default`	`export default function/class/expr`

Named imports (`import { foo, bar } from './module'`) are matched to target file exports to create precise `IMPORTS_SYMBOL` edges — so the graph knows exactly *which symbol* depends on *which symbol*, not just which files.

.kg Format (on disk)

A compact edge-list format in `~/.harness/snapshots/file-tree-snapshot-<hash>.kg`:

```
# Knowledge Graph v2 — D:\Work Projects\Harness
# Nodes: 384  Edges: 512
# Format: n<id>|<type>|<parentId>||<name>|<kind>
#   type: dir|file|symbol

n0|dir|||Harness|
n1|file|n0||README.md|md
n2|dir|n0||server|
n3|file|n2||index.ts|ts
n4|symbol|n3||app|const
n5|file|n2||fileTracking.ts|ts
n6|symbol|n5||getFileTrackingService|function
n7|symbol|n5||FileTrackingService|class

e0|n0|n1|CONTAINS
e47|n5|n6|EXPORTS
e72|n3|n6|IMPORTS_SYMBOL
```

Each line is self-contained — parse with `split("|")`, reconstruct paths by walking parent chains. No JSON overhead, trivially diffable with line-based tools.

`.txt` Visualization (human-readable)

A nested tree with export/import annotations, written alongside the `.kg` file:

```
Harness {
  server {
    index.ts  (exports: app:const; → getFileTrackingService, FileTrackingService)
    fileTracking.ts  (exports: getFileTrackingService:function,
FileTrackingService:class, ...)
    knowledgeGraph.ts  (exports: buildKnowledgeGraph:function,
KnowledgeGraph:interface, ...)
  }
  client {
    src {
      App.tsx  (→ EditorPane)
      panes {
        EditorPane.tsx  (exports: EditorPaneHandle:interface; → FilesPanel,
fileModel)
      }
    }
  }
}
# 124 exports
# 47 file-level imports
# 89 symbol-level imports
```

Filtering

The graph excludes secrets (`.env`), VCS internals (`.git/`), dependencies (`node_modules/`, `vendor/`), build output (`dist/`, `.next/`), IDE caches, binary/media files, and lock files. Project config dotfiles (`.eslintrc.js`, `.prettierrc`, `.editorconfig`) and dot-directories (`.github/`, `.husky/`, `.storybook/`, `.vscode/`) are included.

Groundwork for Graph-Based Reasoning

The knowledge graph is designed as input for graph machine learning and path prediction:

- **One-hop queries**: "What file exports `getFileTrackingService`?" — follow `EXPORTS` backward.
- **Call graph traversal**: `IMPORTS_SYMBOL` edges form a precise dependency graph — follow them to understand data flow.
- **PageRank**: Files imported by many others have higher centrality — identifies core modules.
- **Markov chain path prediction**: Transition probabilities over `IMPORTS_SYMBOL` edges answer "if you just edited symbol X, what file is most likely to need changes next?"
- **GNN input**: Nodes carry features `(type, kind, name)` and edges carry `(type)`. An adjacency matrix can be built directly from the `.kg` file for training graph neural networks on codebase structure.

Comparison with Graphify

Harness and Graphify share the same core idea: pre-build a knowledge graph so AI agents can answer structural questions with a single query instead of scanning raw files. The differences are in scope and design philosophy:

	Harness	Graphify
Trigger	Always-on, built-in IDE feature	Manually invoked CLI skill (`/graphify`)
AST parsing	TypeScript compiler API (TS/JS)	Tree-sitter (23 languages)
LLM involvement	Zero — purely deterministic	Two-pass: deterministic AST + Claude subagents for semantic/concept extraction
Output format	Compact `.kg` edge list (token-optimized) + `.txt` visualization	`.graph.html` (interactive), `.graph.json` (NetworkX), `GRAPH_REPORT.md`
Multimodal	Code files only	Code, PDFs, images, video, audio, diagrams
Community detection	None	Leiden clustering — groups subsystems by edge density
Confidence	N/A (everything is	EXTRACTED / INFERRED /

	Harness	Graphify
tagging	EXTRACTED)	AMBIGUOUS
Query interface	`read_graph` tool — 5 query types (structure, exports, imports_of, exporters_of, dependents)	Python NetworkX API + CLI
Update model	Auto-rebuilds on file watcher events (2s debounce)	SHA256 cache — re-runs only changed files
Agent integration	System prompt rule + tool registry	CLAUDE.md/AGENTS.md rules + PreToolUse hooks (fires before grep/glob)
Footprint	Lightweight, minimal token overhead — always ready	Heavier but richer — HTML visualizations, plain-language reports, multi-format

Harness prioritizes **zero-latency, always-on graph availability** embedded in the IDE loop, with a purpose-built compact format for LLM token efficiency. Graphify prioritizes **depth and breadth** — multi-language, multi-format, semantic reasoning — trading setup time for richer architectural insight.

Security

Harness gives the AI agent access to your filesystem, terminal, and browser. The following mitigations protect against supply-chain risks (compromised API responses, model prompt injection, or malicious tool outputs).

API & Transport

- All DeepSeek API calls use **HTTPS** (`https://api.deepseek.com/v1`).
- Client-entered DeepSeek API keys are stored persistently on disk at `~/.harness/store/api-keys.enc` (AES-256-GCM encrypted), keyed by an HTTP-only session cookie. Keys are never written to browser `localStorage`.
- Agent requests and `/api/models` no longer include the raw key in request bodies or query strings after the initial credential submission.
- The API key is never exposed to child processes (see Terminal Sandbox below).
- `/api/chat/agent/config` exposes only configuration status (`apiKeyConfigured`, `source`), not the key value itself.

- `/api/chat/agent/credentials` is the only route that accepts a raw client-entered key, and it stores that key server-side with file-backed persistence (survives server restarts and app updates).

Tool-Level Guards

Tool	Guard	Blocks
`browser_navigate`	**URL validation**	`javascript:`, `data:`, `file:` protocols. Only `http://` and `https://` allowed.
`run_command`	**Env sanitization**	Any env var whose name contains `KEY`, `SECRET`, `TOKEN`, `PASSWORD`, `CREDENTIAL`, or starts with `npm_` is stripped before the child process starts. Only `PATH`, `HOME`, `USER`, `TEMP`, `SHELL`, `SYSTEMROOT`, `LANG` are forwarded.
`run_in_terminal`	**User permission** + **Command sanitization**	User must explicitly Allow each command before it runs. On Windows, bash syntax (`2>&1`, `&&`) is auto-corrected to PowerShell equivalents.
`read_file` / `grep` / `list_files` / `write_file` / `edit_file` / `delete_file` / `rename_file` / `search_files`	**Secret-file block**	All filesystem tools refuse access to files matching `.env`, `.env.*`, `credentials.*`, `secret.*`, `.pem`, `.key`, `.p12`, `.pfx`, and `config/*secret*` / `config/*key*` paths. These files are also hidden from directory listings and search results.

Browser Sandbox

- The **iframe** is sandboxed with `allow-scripts allow-same-origin allow-forms allow-popups`. Blocked: top-navigation (can't escape the frame), plugins, modals, pointer-lock, downloads.
- A **Content-Security-Policy** header is injected into all proxied pages: `default-src * 'unsafe-inline' 'unsafe-eval' data: blob:; frame-ancestors 'self'; form-action *`. This prevents proxied sites from making `fetch()` calls to Harness's own API endpoints.
- The original site's `X-Frame-Options` and `Content-Security-Policy` headers are stripped to allow framing, but the injected CSP replaces them.

Human-in-the-Loop

Trigger	Mechanism
`run_in_terminal`	Allow/Deny prompt in the agent console
`write_file` / `delete_file`	Accept/Reject undo cards in the agent console

Limitations

- **DeepSeek's API response is still trusted by design.** If the model provider's infrastructure were compromised and injected malicious tool calls, the tool-level guards (URL validation, eval blocking, env sanitization) would catch the most dangerous classes of attack, but not all. Run Harness in isolated environments (VM, dev container) when working with untrusted projects.
- `run_command` **is not container-sandboxed.** It uses `child_process.spawn` with a cleaned environment. For full isolation, run Harness inside Docker or a VM.
- File writes are undoable via the UI, but the agent has full write access to the project directory.

Agent Tools (DeepSeek-powered)

The AI agent has access to these tools when working on your project:

Filesystem

Tool	Description
`read_file`	Read a file with line numbers — always read before editing
`write_file`	Create or overwrite a file with full content (requires user accept/reject)
`edit_file`	Targeted edit by replacing old_string with new_string. Much cheaper — only send the lines that change. old_string must match exactly including whitespace/indentation. Use replace_all to replace all occurrences.
`list_files`	List files and directories in a given path
`search_files`	Recursively find files/folders by name pattern (case-insensitive)
`grep`	Search file contents for a regex pattern — find definitions, usages

Tool	Description
`create_directory`	Create a directory (and parents)
`rename_file`	Rename or move a file or directory
`delete_file`	Delete a file or directory recursively

Terminal

Tool	Type	Description
`run_command`	**Sandbox**	Run a shell command with inline output. Fast, no permission needed. Use for: tests, lint, git, pip, npm, builds, grep. Output is summarized to key lines (errors, warnings, URLs); full output is cached for `read_command_output`.
`run_in_terminal`	**Real terminal**	Run a long-running command in a dedicated terminal tab. User must Allow each command. Use for: `python app.py`, `npm start`, flask, watch mode, interactive shells. The agent **waits for the command to exit or produce recognizable output** (traceback, server-started message, etc.) before receiving the result. Terminal output is captured in full for the UI tool card, and a summarized version (key error/success lines) is sent to the model to save tokens.
`kill_terminal`	**Control**	Kill agent-spawned terminal sessions. `kill_terminal` kills all, `kill_terminal index=N` kills the Nth terminal (0-based, in order of creation). Only kills terminals the agent started — user-created terminals are untouched. Returns a message confirming which terminal was killed and its command. Use to stop servers, free ports, or clean up before finishing.

`run_in_terminal` lifecycle

```
Agent calls run_in_terminal
  → Command sanitized (Windows: strip 2>&1, && → ;)
  → User Allow/Deny prompt
  → Command runs in terminal tab
  → Agent waits for:
       - Process exit (onFinish)
       - Server started pattern (e.g. "listening on :3000")
```

```
      - Error detected (traceback, ModuleNotFoundError, npm ERR!, etc.) → 500ms
flush delay
      - 120s timeout fallback
  → Full terminal output sent to UI tool card
  → Summarized output (key lines, 8 lines / 1200 chars max) pushed to model context
  → Full output cached in commandOutputStore for read_command_output with [cmd #N]
key
  → Agent reads result and acts on errors or proceeds to browser
```

Windows/PowerShell compatibility: On Windows, the terminal runs PowerShell. Bash-isms that would fail silently are auto-corrected:

Bash syntax	Problem	Auto-fix
`2>&1`	PowerShell doesn't understand stderr redirect; causes parse error	Stripped (PowerShell captures stderr natively)
`&&` (chain on success)	PowerShell uses `;` for command chaining	Converted to `;`

Output handling for long logs: If the terminal outputs thousands of lines (e.g. verbose app startup), only a summarized view reaches the model — error lines, warnings, and success markers from the full output, limited to 8 lines / 1200 characters. The full output is always available via `read_command_output cmd_id=N` with pagination (`offset`, `limit`) and regex filtering (`filter`).

Browser

Tool	Scope	Description
Navigation		
`browser_navigate`	Sub-agent only	Navigate to a URL (http/https only). Creates a new browser tab if none exists, or navigates the active tab. Waits for the browser view to mount before returning (up to 2s).
`browser_info`	Sub-agent only	Get current browser tab state: URL, page title, load status, and open tab count.
Observation (read-only)		
`browser_screenshot`	Sub-agent only	Get a page overview: URL, title, and a standardized position-stable grid (`V:WxH`, `XX|N` section headers, `NN|tag#id[type] "label" FLAGS x,y:WxH ^ctx` per line). Capped at 500 elements, 80K chars. On dense pages (>400 shown elements), the

Tool	Scope	Description
		grid auto-splits into horizontal bands (`Band 1/3 y:0–600` etc.) so the agent works one region at a time. Includes visible error text.
`browser_get_dom`	Sub-agent only	Get the full indexed DOM in a standardized position-stable grid. Same rigid field format as `browser_screenshot`. Capped at 3,000 elements, 120K chars. On dense pages, auto-splits into horizontal bands (2 at >400, 3 at >800, 4 at >1,200) with global indices across bands so click/type references remain correct. Indices sorted top-to-bottom, left-to-right (pure geometry). Flags: `A`=clickable, `A+`=interactive, `disabled`, `checked`, `readonly`, `required`. Collapsed/hidden/occluded elements are filtered out.
`browser_console`	Sub-agent only	Get the last 50 console entries (log, warn, error, dialogs) to check for JS errors
`browser_request_errors`	Sub-agent only	Get failed network requests (4xx/5xx/CORS) to verify API calls and resource loads
Interaction (sub-agent only)		
`browser_click`	**Sub-agent only**	Click by DOM index or pixel coordinates. Dispatches full pointer/mouse event sequence.
`browser_type`	**Sub-agent only**	Type text into an input by DOM index — clicks first, clears, then types with realistic keyboard events
`browser_clear`	**Sub-agent only**	Clear the value of an input element by DOM index
`browser_select`	**Sub-agent only**	Select an option from a `<select>` dropdown by value or label
`browser_scroll`	**Sub-**	Scroll the page by pixels or to top/bottom

Tool	Scope	Description
	agent only	
`browser_press_key`	**Sub-agent only**	Press a keyboard key (Enter, Escape, Tab, Arrows, etc.) on the active element
`browser_wait`	**Sub-agent only**	Wait for an element matching a CSS selector to appear (polls every 200ms, default 5s timeout)
Mouse / file upload		
`browser_move_mouse`	**Sub-agent only**	Move the cursor to x,y — triggers hover effects without clicking
`browser_right_click`	**Sub-agent only**	Right-click at x,y — dispatches contextmenu event
`browser_upload_file`	**Sub-agent only**	Set files on a file input by absolute paths

Diagnostics

Tool	Description
`read_problems`	Read current IDE diagnostics — linter errors, TypeScript errors, warnings, hints, debug console, output, browser console. Call after making changes to verify no new errors.
`read_graph`	Query the codebase knowledge graph for structural/dependency information — what a file exports, who imports from a file, which files export a given symbol, the full directory tree. Much faster than grep for dependency questions.

`read_graph` — Knowledge Graph Queries

`read_graph` queries the codebase knowledge graph (see Knowledge Graph for schema details). It reads the `.kg` file from `~/.harness/snapshots/` and answers structural questions without scanning file contents. Use it for dependency analysis, symbol discovery, and project structure exploration.

Query Types

Query	Format	Description	Example
Exports	`exports <file>`	List all symbols exported by a file	`exports server/fileTracking.ts`
Imports of	`imports_of <file>`	List all symbols and files imported by a file	`imports_of client/src/App.tsx`
Exporters of	`exporters_of <symbol>`	Find which files export a symbol with this name	`exporters_of getFileTrackingService`
Dependents	`dependents <file>`	Find which files import from this file (reverse dependency)	`dependents server/fileTracking.ts`
Structure	`structure`	Print the full directory tree (dirs + files, no symbols)	`structure`

Query Details

`exports <file>` — Returns every exported symbol with its kind:

```
server/fileTracking.ts exports:
FileTrackingService:class
getFileTrackingService:function
TrackingMode:type
```

`imports_of <file>` — Returns both symbol-level and file-level imports:

```
client/src/App.tsx imports:
Symbol-level imports:
  EditorPane from client/src/panes/EditorPane.tsx
  StatusBar from client/src/panes/StatusBar.tsx
File-level imports (3):
  client/src/App.css
  client/src/index.css
  client/src/vite-env.d.ts
```

`exporters_of <symbol>` — Case-insensitive symbol search. Useful when you know a function name but not its location:

```
Files exporting 'getFileTrackingService':
server/fileTracking.ts → getFileTrackingService:function
server/index.ts → getFileTrackingService:function
```

`dependents <file>` — Reverse dependency lookup. Shows which files import from a target, including indirect dependents via exported symbols:

```
server/fileTracking.ts is imported by:
client/src/App.tsx
client/src/panes/StatusBar.tsx
server/agent.ts
server/index.ts
```

`structure` — Full directory tree for orientation. Returns sorted paths — no nesting, just one path per line for token efficiency:

```
Directory tree (384 entries):
Harness
Harness/.eslintrc.js
Harness/client
Harness/client/index.html
Harness/client/package.json
...
```

When to Use `read_graph` vs `read_file` vs `grep`

Question	Use	Why
"What does `fileTracking.ts` export?"	`read_graph exports`	Direct lookup — no file scanning
"What is the content of `fileTracking.ts`?"	`read_file`	Content, not structure
"Where is `initFileTracking` called?"	`grep`	Content search across files
"Who imports from `fileTracking.ts`?"	`read_graph dependents`	Reverse dependency — impossible with grep alone
"What files export a function named `foo`?"	`read_graph exporters_of`	Symbol-level query — grep would match comments, strings, calls
"Find all `.ts` files in `server/`"	`list_files` or `search_files`	File/directory listing
"What does this project look like?"	`read_graph structure`	Full tree in one call

Key principle: `read_graph` answers structural questions (what exists, how things connect). `read_file` and `grep` answer content questions (what's inside, where is it used). When unsure, prefer `read_graph` for dependency/export queries — it's a single call vs potentially dozens of grep searches.

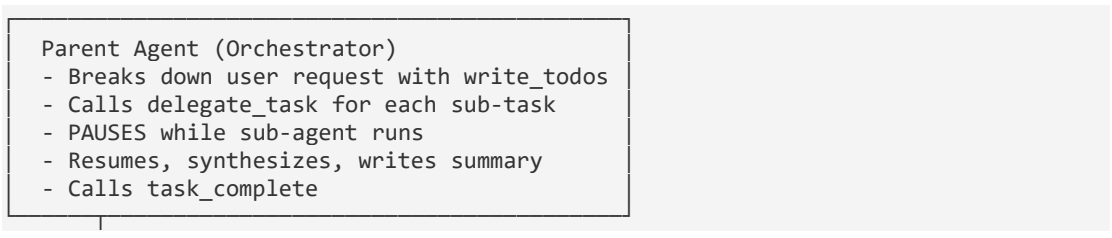
Control

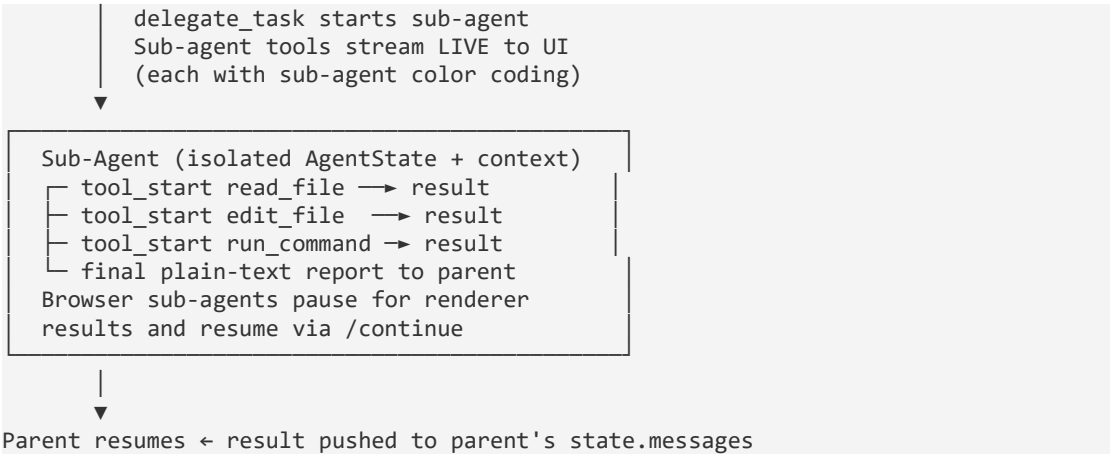
Tool	Description
`write_todos`	Create or update a structured task list to track progress. In step-by-step mode, this is the ONLY tool available during planning — the agent must create a complete plan before any execution begins.
`write_summary`	Write the final **structured summary** using the template: `### Changes Made`, `### Verification`, `### Outcome`. Vague summaries are **rejected**. If `write_todos` was used: the summary must also include a `### Todo Progress` section listing each item's final status. On success, the UI renders the summary once in `agent-body` as markdown preview instead of as a tool card.
`task_complete`	Finalize the run. Has no parameters and is **rejected unless** `write_summary` **has been called**. Also rejected if any todo items are still pending/in_progress. The SSE `done` reply reuses the stored summary, but the client dedupes it so the final summary is not rendered twice.
`delegate_task`	Delegate a sub-task to a specialized sub-agent (browser, code-search, code-writer, researcher, planner, frontend-specialist, backend-specialist, security-auditor, architect-analyst, docs-analyst, documentation-writer) that runs independently with its own context window. Sub-agents run sequentially — each must complete before the next starts.

Multi-Agent Delegation

Harness supports **sub-agent delegation** — the main agent can spawn specialized sub-agents to handle complex sub-tasks in isolation. Each sub-agent gets its own context window, so its conversation history does not bloat the parent agent's context. Sub-agents run **sequentially** — each must complete before the next starts, managing RAM usage.

Architecture





Agent Profiles

Profile	Tools	Iterations	Description
`browser`	`browser_navigate`, `browser_info`, `browser_screenshot`, `browser_get_dom`, `browser_click`, `browser_type`, `browser_clear`, `browser_select`, `browser_press_key`, `browser_console`, `browser_request_errors`, `browser_scroll`, `browser_wait`, `browser_move_mouse`, `browser_right_click`, `browser_upload_file`	100	Full browser automation — unlimited turns for intensive testing. Navigates, clicks, types, scrolls, fills forms, inspects DOM/console/network.
`code-search`	`read_file`, `list_files`, `search_files`, `grep`, `read_graph`	20	Read-only code exploration. Finds files, reads code, reports findings. Never edits.
`code-writer`	Full filesystem + `run_command`, `read_problems`, `read_graph`	50	Implements features or fixes bugs. Reads, edits, builds, and verifies.
`researcher`	`read_file`, `list_files`, `search_files`, `grep`,	25	Explores codebase to answer questions.

Profile	Tools	Iterations	Description
	`run_command`, `read_graph`		Reports with file paths and line numbers.
`planner`	`read_file`, `list_files`, `search_files`, `grep`, `read_graph`, `write_todos`	25	Analyzes project and creates structured step-by-step plans. Outputs a todo list with ordered, actionable steps.
`frontend-specialist`	Full filesystem + `run_command`, `read_problems`, `read_graph`, `browser_screenshot`, `browser_get_dom`, `browser_console`, `browser_request_errors`	50	Implements UI features and components. Visually verifies changes in the browser.
`backend-specialist`	Full filesystem + `run_command`, `read_problems`, `read_graph`	50	Implements API routes, services, and database logic. Focuses on server-side patterns and data integrity.
`security-auditor`	`read_file`, `list_files`, `search_files`, `grep`, `run_command`, `read_graph`, `read_problems`	30	Audits code for vulnerabilities. Runs security scans, reports findings with severity and remediation. Never edits.
`architect-analyst`	`read_file`, `list_files`, `search_files`, `grep`, `read_graph`	25	Analyzes project architecture, dependency graphs, and module structure. Reports architectural concerns and recommendations. Never edits.
`docs-analyst`	`read_file`, `list_files`, `search_files`, `grep`, `read_graph`	20	Audits documentation coverage and quality. Identifies gaps and outdated docs. Never

Profile	Tools	Iterations	Description
			edits.
`documentation-writer`	`read_file`, `write_file`, `edit_file`, `list_files`, `search_files`, `grep`, `read_graph`, `create_directory`	30	Creates or improves documentation. Writes README sections, API docs, and guides.

Key Design

Feature	Detail
Context isolation	Each sub-agent has its own `AgentState` — messages do not pollute the parent's context
Tool allowlisting	Sub-agents receive only the tools their profile specifies (e.g. code-search can never write files)
Headless execution	All non-browser sub-agents run entirely server-side — no browser or terminal tools. Frontend-specialist has read-only browser tools for visual verification.
Browser delegation	The parent agent has NO browser tools — not even read-only ones. ALL browser interaction (navigating, inspecting DOM, taking screenshots, checking console/network, clicking, typing, scrolling) goes through the browser sub-agent via `delegate_task agent_type: "browser"`. This keeps the main agent's context clean and forces structured delegation.
Live streaming	Sub-agent tool calls stream live to the UI as colored tool cards in real-time. Parent appears paused during delegation. Sub-agent text events are filtered — only tool_start/tool_end cards are shown, preventing message pollution.
Result summarization	Sub-agent results are compressed before returning to the parent, preserving context budget
Parallelism	Not supported — sub-agents run sequentially. Each must complete before the next starts, managing RAM usage. The agent should call `delegate_task` multiple times for independent sub-tasks.
Color coding	Every tool card has a left-border color: blue (main), teal (browser), green (code-search), amber (code-writer), purple

Feature	Detail
	(researcher), indigo (planner), cyan (frontend-specialist), blue (backend-specialist), red (security-auditor), orange (architect-analyst), lime (docs-analyst), pink (documentation-writer). Makes it easy to identify which agent executed each tool call.
Agent footer	Shows "Completed" when the conversation finishes normally (SSE `done` event). Shows "Stopped" only on errors or a 5-minute safety timeout. The footer label corresponds strictly to SSE stream state — not app focus.
Background operation	The SSE stream uses `fetch`-based streaming — operates independently of window focus. Agent conversations continue in background with no interruption when the app is minimized or behind other windows.

Usage

The parent agent uses these tools just like any other:

```
Agent: write_todos todos=[
  {id:1 text:"Research existing auth code" status:pending},
  {id:2 text:"Add login endpoint" status:pending}
]

Agent: delegate_task task="Find all authentication-related code
  in the project. Report file paths, line numbers, and patterns used."
  agent_type="code-search"

→ [Code Search Agent] Completed in 4 turns.
  Found auth code in:
  - server/auth.ts:45-120 (JWT verification, password hashing)
  - client/src/Login.tsx:1-80 (login form component)
  ...
```

Browser agent example — the sub-agent can now navigate, click, type, and inspect pages interactively:

```
Parent: delegate_task task="Go to http://localhost:3000/login,
  type 'admin' into the email field, type 'pass123' into the
  password field, click Sign In, and report what happens."
  agent_type="browser"

→ [Browser Agent] browser_navigate http://localhost:3000/login
  → Renderer executes → page loads
→ [Browser Agent] browser_get_dom
  → Renderer returns indexed elements in standardized
    position-stable grid (V:WxH, XX|N sections, A/A+ flags)
    On dense pages, splits into bands like "Band 1/2 y:0-450"
    so the agent reads one region at a time.
→ [Browser Agent] browser_click index=12  (email input with A+ flag)
  → Renderer clicks → input focused
→ [Browser Agent] browser_type index=12 text="admin"
```

```
  → Renderer types → "admin" entered
→ [Browser Agent] browser_click index=15  (password input)
→ [Browser Agent] browser_type index=15 text="pass123"
→ [Browser Agent] browser_click index=18  (Sign In button with A+ flag)
→ [Browser Agent] browser_screenshot
  → Renderer returns URL, title, banded grid
    (auto-splits into regions if viewport is dense),
    and filtered error text
  → Sees "Welcome, admin!" in Band 2, Middle-Center section

→ [Browser Agent] Completed in 10 turns.
  Login test: SUCCESS. Navigated to login page,
  filled email and password fields, clicked Sign In.
  Result: "Welcome, admin!" displayed.
```

Parallel example — code-writer + researcher run simultaneously:

```
Agent: delegate_parallel tasks=[
  {task:"Implement POST /api/login in server/auth.ts", agent_type:"code-writer"},
  {task:"Research how existing API routes handle error responses",
agent_type:"researcher"}
]

→ 2 sub-agents completed.
  [1] Wrote ~500 tokens to server/auth.ts. Build passed.
  [2] Found error handling pattern in server/middleware.ts:30-55...
```

When to use

- `delegate_task`: For complex sub-tasks that would take many turns (deep research, feature implementation, multi-file refactoring, browser testing). For multiple independent sub-tasks, call `delegate_task` sequentially — each completes before the next starts.

IDE-Driven Step-by-Step Execution (Force Todo)

In the default agent loop, the LLM controls when to create, update, and complete todos — it can skip steps, forget updates, or jump ahead. **Step-by-step mode** inverts this: the **IDE/server locks the todo list** and forces the agent through each step one at a time via isolated sub-agents.

How it works

```
User sends task → POST /api/chat/agent/stream/stepbystep

Phase 1: PLANNING
  Agent only has write_todos — no other tools
  Agent MUST create a complete, ordered plan
  → Server validates (non-empty, specific steps)

Phase 2: LOCKED
  Server locks the todo list → SSE "step_plan" event
  UI renders the locked plan in the pending banner

Phase 3: EXECUTE (per step)
  For each pending todo:
```

```
    → SSE "step_begin" event
    → Code-writer sub-agent spawned with ONLY this step's context
    → Sub-agent has full filesystem + run_command tools (max 50 iters)
    → Streaming tool_start/tool_end events shown in UI
    → Sub-agent returns a final plain-text report → SSE "step_end" event
    → Step marked completed/failed, next step begins
  Previous step results are passed as context to subsequent steps

Phase 4: WRAP-UP
  → SSE "done" event with allStepResults summary
```

Key differences from default mode

Aspect	Default Mode	Step-by-Step Mode
Planning	Agent can plan AND execute in same turn	Strict separation: plan first, execute later
Todo ownership	Agent-driven — LLM chooses when to update	IDE-driven — server locks todos, forces progression
Execution	Agent works on anything at any time	One step at a time, isolated sub-agent per step
Context	Full conversation history in one loop	Each step gets fresh sub-agent with only that step + previous results
Tool restriction	Full tool set available	Planning: only write_todos. Execution: code-writer tools (no browser/terminal)
Max turns	50 per agent loop	Planning: 5 turns. Per step: 50 turns (configurable per agent profile)

Why use it

- **Deterministic execution**: The server enforces todo progression — agent can't skip or forget steps
- **Isolated failures**: If a step's sub-agent fails, subsequent steps still run with clear failure context
- **Clean context per step**: Each step sub-agent starts fresh, avoiding context bloat from earlier steps
- **Verifiable progress**: The UI shows a locked plan with per-step status (pending → in_progress → completed/failed)

Summary Lock

Harness enforces **structured summaries** on `write_summary`. The server validates every summary against a required template, and rejects `task_complete` unless `write_summary` has been called.

```
### Changes Made
- [file path]: [what was changed]
### Verification
- [build/test/check result]
### Outcome
- [concise description of what was accomplished]
```

Summaries that are too short, match thought-process patterns (e.g. "I did the task", "OK, completed"), or lack concrete details (no file references, actions, or results) are **rejected**. The agent receives the rejection as a tool error and must call `write_summary` again with a proper summary.

Persistent Memory

Harness includes a **cross-session memory system** backed by SQLite. The agent can store key decisions, user preferences, project conventions, and discovered patterns — and recall them in future sessions.

How it works

```
Agent detects an important fact
  → calls remember(key, value, category, tags)
  → value is optionally embedded via DeepSeek embeddings API
  → stored in ~/.harness/store/memory.db (SQLite, global user store)

Next session:
  → Agent calls recall(query: "UI framework")
  → Semantic search (cosine similarity) if embeddings exist
  → Falls back to keyword search if embedding API unavailable
  → Returns ranked results with scores
```

Tools

Tool	Description
`remember`	Store a key decision, user preference, project convention, or important fact. Persists across sessions in SQLite. Categories: `decision`, `preference`, `convention`, `fact`, `pattern`, `general`. Tags help group related memories. The value is optionally embedded via DeepSeek API for semantic search.
`recall`	Search stored memories by semantic meaning or exact key. If embeddings are available, uses cosine similarity search. Otherwise falls back to keyword matching on key, value, tags, and category. Pass no params to list all memories.
`forget`	Remove a stored memory by its key. Use when a decision is

Tool	Description
	reversed, a preference changes, or stored information becomes outdated.

Storage

Detail	Value
Database	SQLite (WAL mode) at `~/.harness/store/memory.db` (global, not in project dir)
API Keys	AES-256-GCM encrypted file at `~/.harness/store/api-keys.enc` (persistent, survives restarts and app updates)
Client State	JSON file at `~/.harness/store/client-state.json` — mirrors all browser `localStorage` data (model, chat history, recent paths, open tabs, presets, terminal history) so it survives reinstalls
Schema	`id`, `key` (unique), `value`, `category`, `tags`, `embedding` (BLOB), `created_at`, `updated_at`
Embeddings	Generated via DeepSeek `/v1/embeddings` endpoint (optional; graceful fallback to keyword search if unavailable)
Retrieval	Embedding cosine similarity search → keyword `LIKE` fallback → list-all

When the agent uses memory

- **Proactive storage**: When the user says "let's use X", "I prefer Y", or establishes a project convention, the agent calls `remember` without being asked.
- **Session startup**: The agent is instructed to `recall` relevant memories at the start of a task to pick up past decisions and preferences.
- **Memory cleanup**: When preferences change or decisions are reversed, the agent can `forget` outdated entries.

Files

File	Role
`server/memory.ts`	`MemoryStore` class — SQLite CRUD, embedding search, cosine similarity, global singleton
`server/deepseek.ts`	`generateEmbedding()` — calls DeepSeek embeddings API, returns `Float32Array`

File	Role
`server/agent.ts`	`runMemoryTool()` — tool execution handler; wired in both `agentLoop()` and `agentLoopStream()`

Agent Command Catalog

Every shell command the agent can potentially issue via `run_command` or `run_in_terminal`. These are extracted from the agent's system prompt chunks and the `detectProjectBuild()` auto-detection logic in [server/agent.ts](file:///d:/Work Projects/Harness/server/agent.ts).

JavaScript / TypeScript

Command	Usage	Source
`npx tsc --noEmit`	Type-check all files (preferred)	`LANG_JS` + `detectProjectBuild`
`npm run build`	Full build via package.json scripts	`LANG_JS` + `detectProjectBuild`
`npx eslint .`	Lint all files	`LANG_JS`
`npm install`	Install all project dependencies	`LANG_JS`
`npm install <pkg>`	Install a specific package	`LANG_JS`

Python

Command	Usage	Source
`python -m py_compile <file>.py`	Single-file syntax check	`LANG_PYTHON`
`python -m compileall .`	Syntax check all .py files	`LANG_PYTHON` + `detectProjectBuild`
`python -m pytest`	Run tests	`LANG_PYTHON`
`pip install -r requirements.txt`	Install all project dependencies	`LANG_PYTHON`
`pip install <pkg>`	Install a single package	`LANG_PYTHON` + `SERVER_STARTUP`

Go

Command	Usage	Source
`go build ./...`	Compile all packages	`LANG_GO`
`go vet ./...`	Static analysis	`LANG_GO` + `detectProjectBuild`
`go test ./...`	Run all tests	`LANG_GO`

Rust

Command	Usage	Source
`cargo check`	Fast compile check (no binary)	`LANG_RUST` + `detectProjectBuild`
`cargo build`	Full compilation	`LANG_RUST`
`cargo test`	Run tests	`LANG_RUST`
`cargo clippy`	Lint with extra warnings	`LANG_RUST`

Java

Command	Usage	Source
`mvn compile`	Maven build	`LANG_JAVA` + `detectProjectBuild`
`gradle build`	Gradle build	`LANG_JAVA`
`gradle compileJava`	Gradle compile only	`detectProjectBuild`
`javac <File>.java`	Single file (no build tool)	`LANG_JAVA`

C / C++

Command	Usage	Source
`gcc -Wall -Wextra <file>.c -o output`	Single C file with warnings	`LANG_C`
`g++ -Wall -Wextra <file>.cpp -o output`	Single C++ file with warnings	`LANG_C`
`cmake --build build`	CMake projects	`LANG_C` + `detectProjectBuild`
`make`	Makefile projects	`LANG_C` +

Command	Usage	Source
		`detectProjectBuild`

Ruby

Command	Usage	Source
`ruby -c <file>.rb`	Syntax check (safe, no execution)	`LANG_RUBY` + `detectProjectBuild`
`bundle exec rake test`	Run tests via Rake	`LANG_RUBY`
`bundle exec rspec`	Run RSpec tests	`LANG_RUBY`
`bundle install`	Install gem dependencies	`LANG_RUBY`
`gem install <pkg>`	Install a single gem	`LANG_RUBY`

PHP

Command	Usage	Source
`php -l <file>.php`	Single file syntax lint	`LANG_PHP`
`php -l *.php`	Lint all PHP files	`LANG_PHP` + `detectProjectBuild`
`composer install`	Install dependencies	`LANG_PHP`

Shell (Bash)

Command	Usage	Source
`bash -n <script>.sh`	Syntax check without executing	`LANG_SHELL`
`shellcheck <script>.sh`	Static analysis (if installed)	`LANG_SHELL`

Cross-language / Generic

Command	Usage	Source
`git status --porcelain -u`	Staged/unstaged file tracking	Server SCM API
`git log --max-count=20`	Recent commit history	Server SCM API
`git diff -- <file>`	Show unstaged changes for a file	Server SCM API

Command	Usage	Source
`git fetch --all`	Fetch from all remotes	Server SCM API
`git pull`	Pull latest changes	Server SCM API
`git push`	Push local commits	Server SCM API

Project auto-detection (`read_problems`)

When the agent calls `read_problems`, the server auto-detects the project type and runs:

Detection signal	Auto-command
`Cargo.toml` exists	`cargo check 2>&1`
`go.mod` exists	`go vet ./... 2>&1`
`pom.xml` exists	`mvn compile 2>&1`
`build.gradle` or `build.gradle.kts` exists	`gradle compileJava 2>&1`
`package.json` + `tsconfig.json`	`npx tsc --noEmit 2>&1`
`package.json` with build script	`npm run build 2>&1`
`package.json` (no tsconfig, no build script)	`npx tsc --noEmit 2>&1`
`requirements.txt` / `pyproject.toml` / `setup.py` / `.py` files	`python -m compileall . 2>&1`
`Gemfile` exists	`ruby -c *.rb 2>&1`
`composer.json` exists	`php -l *.php 2>&1`
`Makefile` exists	`make 2>&1`
`CMakeLists.txt` exists	`cmake --build build 2>&1`
None of the above	`npx tsc --noEmit` / `python -m compileall .` / `go vet ./...` (general suggestion)

Server-specific (via `run_in_terminal`)

Commands the agent is instructed to launch in a real terminal tab:

Framework	Typical command	Mentioned in
Python (generic)	`python app.py` / `python server.py`	`run_command` block list
Flask	`flask run`	`run_command` block list
Django	`python manage.py runserver`	`run_command` block list
FastAPI	`uvicorn main:app`	`run_command` block list
Gunicorn	`gunicorn app:app`	`run_command` block list
Node.js (Express)	`node server.js`	`run_command` block list
npm scripts	`npm start` / `npm run dev`	`run_command` block list
Next.js	`next dev` / `next start`	`run_command` block list
Vite	`vite`	`run_command` block list
Go	`go run .`	`run_command` block list
Rust	`cargo run`	`run_command` block list
Webpack	`webpack-dev-server`	`run_command` block list
npx runners	`npx serve`, `npx vite`, `npx next`	`run_command` block list

> Note: These server commands are BLOCKED in `run_command` and redirected to `run_in_terminal`. The agent is explicitly told to use `run_in_terminal` for all server start commands.

Troubleshooting by Language

The agent knows how to diagnose and fix errors for each language stack. Below is the guidance it follows — useful to understand what the agent will do when your build fails.

JavaScript / TypeScript

Scenario	Tool	Command / Approach
Type-check	`run_command`	`npx tsc --noEmit`
Full build	`run_command`	`npm run build` (check `package.json` first)
Lint only	`run_command`	`npx eslint .`
Missing module	`run_command`	`npm install <pkg>`

Scenario	Tool	Command / Approach
Runtime errors in browser	`browser_console`	After starting dev server, check console output
Failed API calls	`browser_request_errors`	Check for 404/500/CORS errors in the browser
Find a definition	`grep`	Regex search across project files

Python

Scenario	Tool	Command / Approach
Syntax check (single file)	`run_command`	`python -m py_compile <file>.py`
Syntax check (all files)	`run_command`	`python -m compileall .`
Run tests	`run_command`	`python -m pytest`
Install dependencies	`run_command`	`pip install -r requirements.txt` or `pip install <pkg>`
Flask/Django runtime errors	`browser_screenshot` or `browser_get_dom`	Flask debug mode shows full tracebacks in the browser; the standardized grid with position buckets and `A`/`A+` flags helps distinguish nav chrome from the actual error pane
HTTP errors from backend	`browser_request_errors`	Check for 500 errors and CORS issues
Find where a function is defined	`grep`	`def <name>` or `class <Name>`
Read stack traces	`read_file`	Open the failing file at the line from the traceback

Go

Scenario	Tool	Command / Approach
Compile check	`run_command`	`go build ./...`

Scenario	Tool	Command / Approach
Static analysis	`run_command`	`go vet ./...`
Run tests	`run_command`	`go test ./...`
Unused import	`edit_file`	Remove the import line (Go forbids unused imports)
Find definitions	`grep`	`func <Name>` or `type <Name>`

Rust

Scenario	Tool	Command / Approach
Fast compile check	`run_command`	`cargo check` (preferred — no binary output)
Full build	`run_command`	`cargo build`
Lint	`run_command`	`cargo clippy`
Run tests	`run_command`	`cargo test`

Java

Scenario	Tool	Command / Approach
Maven compile	`run_command`	`mvn compile`
Gradle build	`run_command`	`gradle build`
Single file compile	`run_command`	`javac <File>.java`
Find class definition	`grep`	`class <Name>`

C / C++

Scenario	Tool	Command / Approach
Compile with warnings	`run_command`	`gcc -Wall -Wextra <file>.c -o output`
CMake build	`run_command`	`cmake --build build`
Make build	`run_command`	`make`
Find function definition	`grep`	`void <name>(` or `int <name>(`

Ruby

Scenario	Tool	Command / Approach
Syntax check	`run_command`	`ruby -c <file>.rb`
Install deps	`run_command`	`bundle install`
Run tests	`run_command`	`bundle exec rspec` or `bundle exec rake test`

PHP

Scenario	Tool	Command / Approach
Syntax lint	`run_command`	`php -l <file>.php`
Install deps	`run_command`	`composer install`

Shell (Bash)

Scenario	Tool	Command / Approach
Syntax check	`run_command`	`bash -n <script>.sh`
Static analysis	`run_command`	`shellcheck <script>.sh`

General troubleshooting flow

1. **Start the server** (`run_in_terminal`) — user must Allow
2. **Check for build errors** (`run_command`) — fixes go through `edit_file` / `write_file`
3. **Verify the page loads** (`browser_info` → `browser_screenshot` / `browser_get_dom`, then use the standardized grid, position buckets, and `A`/`A+` flags to target the right element)
4. **Check browser runtime errors** (`browser_console`, `browser_request_errors`)
5. **Read relevant source files** (`read_file`) before making fixes
6. **Make targeted edits** (`edit_file` — just send the lines that change)
7. **Rebuild and verify** — repeat until clean

Avoiding tool hallucinations

The agent works with a fixed tool registry. To prevent it from inventing tools that don't exist:

- **Reading files** → use `read_file` (never `cat`, `head`, `tail`)
- **Listing directories** → use `list_files` (never `ls`, `dir`)
- **Finding files by name** → use `search_files` (never `find`, `locate`)
- **Searching file contents** → use `grep` (the tool, not the shell command)
- **Editing files** → use `edit_file` (never `sed`, `awk`)

- **Writing files** → use `write_file` (never `echo >`, `cp`)
- **Running commands** → use `run_command` for short tasks, `run_in_terminal` for servers (never background with `&` or `nohup`)
- **Checking diagnostics** → use `read_problems` (not `tsc`, `eslint`, or `pylint` directly — those go through `run_command`)
- **Dependency/structural queries** → use `read_graph` (what exports X? who imports from Y?) — much faster than grep for these
- **Starting servers** → use `run_in_terminal` only (never `run_command` for `python app.py`, `npm start`, etc.)

MCP (Model Context Protocol)

Harness can act as an **MCP server**, exposing its filesystem, terminal, git, and system tools to any MCP-compatible client (Claude Desktop, Cursor, VS Code with Copilot, etc.).

Which transport to use

The configuration depends on how you're running Harness:

Scenario	Transport	Why
Development (source checkout)	Stdio or SSE	Both work; stdio gives you project isolation
Electron desktop app (packaged)	**SSE only**	The Express server already runs inside Electron — no extra process needed

> In an Electron app: the Harness Express server starts inside the Electron main process. The MCP endpoints (`/api/mcp`, `/api/mcp/sse`) are available automatically on the server's assigned port. You do NOT need a separate process or a `cwd` pointing to the source code — just connect via SSE.

Development mode (source checkout)

When running Harness from source (`npm run dev`), you have both options:

Option A: SSE (simplest — no extra config)

Start the server, then point any MCP client at the running endpoint (check the console output for the port, or read `~/.harness/ports/express-port`):

```
{
```

```
  "mcpServers": {
    "harness": {
      "url": "http://localhost:<port>/api/mcp/sse"
    }
  }
}
```

Works with Claude Desktop, Cursor, VS Code, and any SSE-compatible client.

Option B: Stdio (project isolation)

Run a separate process per project. The `cwd` points to the Harness source checkout so `tsx` and the server files are found:

```
npx tsx server/mcp-server.ts "D:\my-project"
```

Claude Desktop config (`%APPDATA%\Claude\claude_desktop_config.json`):

```
{
  "mcpServers": {
    "harness": {
      "command": "npx",
      "args": ["tsx", "server/mcp-server.ts", "D:\\my-project"],
      "cwd": "D:\\Work Projects\\Harness"
    }
  }
}
```

Cursor config (`Settings > MCP > Add Server`):

```
{
  "mcpServers": {
    "harness": {
      "command": "npx",
      "args": ["tsx", "server/mcp-server.ts", "${workspaceFolder}"],
      "cwd": "D:\\Work Projects\\Harness"
    }
  }
}
```

Electron desktop app (packaged)

When Harness is installed as a desktop app, the server starts automatically. The port is written to `~/.harness/ports/express-port`. Use SSE transport only — no `command`/`cwd` needed:

```
{
  "mcpServers": {
    "harness": {
      "url": "http://localhost:<port>/api/mcp/sse"
    }
  }
}
```

The embedded Express server handles all MCP requests. The project root is automatically set to the currently open project folder in the Harness UI, so tools like `read_file`, `grep`, and `run_command` operate on the right project automatically.

Why stdio mode doesn't work well for packaged Electron apps:

- There's no `tsx` runtime on the user's machine
- The source files (`server/mcp-server.ts`) are compiled/bundled, not on disk
- The Express server is already running inside Electron — spawning a second process is redundant

If you really need stdio from a packaged app, you can compile the MCP entry point to a standalone `.cjs` file and bundle it with the app. But SSE is the intended path.

MCP tools

The following tools are exposed via MCP:

Tool	Description
`read_file`	Read a file with line numbers or list a directory
`write_file`	Create or overwrite a file
`edit_file`	Targeted string replacement in a file
`list_files`	List files and directories at a path
`search_files`	Find files/folders by name pattern
`grep`	Search file contents with regex
`run_command`	Execute a shell command (sandboxed, no permission needed)
`create_directory`	Create a directory (and parent dirs)
`delete_file`	Delete a file or directory (recursive)
`rename_file`	Rename or move a file or directory
`git_status`	Get staged and unstaged git changes, current branch
`git_log`	Get recent commit history
`git_diff`	Get the diff for a specific file
`system_info`	Get CPU, memory, disk, OS details

Protocol details

Harness implements **MCP protocol version** `2024-11-05` with JSON-RPC 2.0:

1. **Initialize** — Client sends `initialize` → Server returns capabilities and server info
2. **List tools** — Client sends `tools/list` → Server returns tool definitions with JSON Schema
3. **Call tool** — Client sends `tools/call` → Server executes the tool and returns `{ content: [{ type: "text", text: "..." }] }`

The server only exposes **tools** capability — no resources or prompts.

Example MCP exchange

```
→ {"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-
11-05","capabilities":{},"clientInfo":{"name":"test","version":"1.0"}}}
← {"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2024-11-
05","capabilities":{"tools":{}},"serverInfo":{"name":"harness","version":"1.0.0"}}}

→ {"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
← {"jsonrpc":"2.0","id":2,"result":{"tools":[...]}}

→
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"read_file","argumen
ts":{"path":"server/index.ts","limit":10}}}
← {"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text","text":"   1| import
\"dotenv/config\";\n..."}],"isError":false}}
```

Project Structure

```
Harness/
├── .env.example                    # Template for environment variables (DeepSeek
API key)
├── package.json                    # Root deps (Express, better-sqlite3, node-
pty), scripts (dev, test, desktop:build)
├── package-lock.json
├── tsconfig.json                    # TypeScript config for server (ES2022, strict,
vitest globals)
├── vitest.config.ts                # Vitest runner config (node env, 30s timeout)
├── README.md                       # You're reading it
│
├── build/
│   └── icon assets (ico, png, svg)  # Electron app icons
│
├── scripts/
│   ├── embed-icon.js               # Embeds icon into the Electron executable
│   └── generate-icon.js            # Generates icon from SVG source
│
├── server/
│   ├── index.ts                    # Express server entry: API routes, WebSocket
terminal, agent SSE streaming, MCP, browser proxy, system stats
│   ├── agent.ts                    # Agent core: 27 tool definitions, 11 sub-
agent profiles, delegate_task, agentLoop/Stream/StepByStep, permission gating, ITR
system prompt builder, history compaction
│   ├── deepseek.ts                 # DeepSeek API client: blocking + streaming
chat w/ tool-calling, embeddings, KV cache tracking, usage reporting
│   ├── terminalManager.ts          # Terminal session manager: PTY (node-pty) and
pipe fallback, WebSocket I/O, venv auto-activation, localhost URL detection
```

```
│   ├── lsp.ts                       # LSP client: spawns language servers (pyright,
gopls, etc.), SSE diagnostic streaming, 30+ languages
│   ├── mcp.ts                       # MCP server: JSON-RPC handler, tool set for
external AI clients, stdio + SSE transport
│   ├── mcp-server.ts                # Standalone MCP server entry point (stdio
mode)
│   ├── memory.ts                    # SQLite persistent memory store
(~/.harness/store/memory.db): keyword + embedding search, used by agent
remember/recall/forget tools
│   ├── cryptoStore.ts               # AES-256-GCM encrypted API key storage
(~/.harness/store/api-keys.enc)
│   ├── harnessPaths.ts              # Centralized path resolution for ~/.harness/
directory structure
│   ├── fileTracking.ts              # Smart file tracking: auto-detects Git vs
fs.watch watcher mode, mid-session Git detection, snapshot/patch file tree context
│   ├── fileTrackingStore.ts         # JSON-backed file metadata cache
(~/.harness/store/file-tracking.json) for watcher mode
│   ├── knowledgeGraph.ts            # Codebase knowledge graph builder: dir/file
nodes, CONTAINS + IMPORTS edges, .kg serialization, visualization
│   └── __tests__/
│       ├── agent.tooldefs.test.ts    # Tool definition schema validation
│       ├── agent.fs.test.ts          # Filesystem tool execution tests
│       ├── agent.command.test.ts     # Command execution tests
│       ├── agent.loop.test.ts        # Agent loop integration tests
│       └── api.test.ts              # API endpoint integration tests
│
├── client/
│   ├── package.json                 # Client deps (React 18, Monaco, xterm.js),
Vite + TypeScript
│   ├── vite.config.ts               # Vite dev config: reads Express port from
~/.harness/ports/express-port, proxies /api + /ws + /_browser
│   ├── tsconfig.json                # Client TypeScript config (ES2020, DOM,
react-jsx)
│   ├── index.html                   # SPA entry: mounts React app in <div
id="root">
│   ├── public/
│   │   └── icon.svg                 # App icon SVG
│   └── src/
│       ├── main.tsx                 # React DOM entry: renders <App />
│       ├── App.tsx                   # Root component: folder picker, session state,
resizable layout (editor + agent console)
│       ├── App.css                  # Global dark-theme styles: pane layout,
editor chrome, agent cards, sub-agent color coding (11 agent types), welcome screen
│       ├── electron.d.ts            # Type declarations for window.harnessDesktop
bridge and <webview> JSX
│       ├── vite-env.d.ts            # Vite client type declarations
│       ├── stateSync.ts             # Client state persistence: mirrors all
localStorage to ~/.harness/store/client-state.json (survives reinstalls)
│       ├── panes/
│       │   ├── EditorPane.tsx       # Main editor: Monaco tabs, file tree, browser
tab strip, terminal, menu bar, status bar
│       │   ├── AgentConsole.tsx     # Agent chat UI: streaming messages, diff
previews, permission prompts, tool cards with agent color coding, markdown
rendering
│       │   ├── TerminalPane.tsx     # xterm.js terminal: WebSocket-backed PTY,
Ctrl+click links, scrollback, agent bridge
│       │   ├── FilesPanel.tsx       # File explorer tree: virtual files + backend
FsEntry, create/rename/delete
│       │   ├── BrowserView.tsx      # Embedded browser: iframe proxy,
getIndexedDom/clickElement/typeIntoElement agent APIs, DOM indexing helpers
│       │   ├── MenuBar.tsx          # Dropdown menus: File, Edit, View, Run, Help
with keyboard shortcuts
│       │   ├── StatusBar.tsx        # Status bar: cursor position, encoding,
indent, language, LSP errors, memory count
│       │   ├── ScmPanel.tsx         # Source control: git status, commit log,
fetch/pull/push, diff
```

```
│       │   ├── NameDialog.tsx       # Modal dialog for create/rename files and
folders
│       │   ├── PathDialog.tsx       # Modal dialog for manually opening a folder
path
│       │   ├── AgentTerminalBridge.ts # Bridge: agent commands → real terminal
execution
│       │   ├── fileModel.ts         # VFile type, detectLanguage(), file/folder
icon helpers
│       │   └── browserFs.ts         # Browser File System API:
pickAndEnumerateFolder, readFile, writeFile
│       └── hooks/
│           └── useResizable.tsx     # Drag-to-resize panel splitter hook
│
└── electron/
    ├── main.cjs                      # Electron main process: BrowserWindow,
server lifecycle, IPC (folder/file picker, geo, permissions), browser session
    ├── preload.cjs                   # Preload bridge: exposes
window.harnessDesktop (openFolder, openFile, onBrowserOpenUrl, setSitePermissions)
    ├── browser-preload.cjs           # Browser webview preload: geolocation bridge,
_blank link interception
    └── native-location.cjs           # Windows geolocation via PowerShell
GeoCoordinateWatcher
```

Testing

Harness includes an automated test suite using Vitest. Tests cover all agent tools, the agent loop (with mocked DeepSeek API), tool schema validation, and API endpoints.

Running tests

```
# Run all tests once
npm test

# Run tests in watch mode (re-run on file changes)
npm run test:watch

# Run tests with coverage report
npm run test:coverage

# Run a specific test file
npx vitest run server/__tests__/agent.fs.test.ts
```

Test structure

```
server/__tests__/
├── agent.fs.test.ts          # Filesystem tools: read_file, write_file, edit_file,
│                             #   list_files, search_files, grep, create_directory,
│                             #   delete_file, rename_file, write_todos (34 tests)
├── agent.command.test.ts     # Command tools: run_command, run_in_terminal,
│                             #   read_command_output (19 tests)
├── agent.loop.test.ts        # Blocking and streaming agent loops with mocked
│                             #   DeepSeek API responses (14 tests)
├── agent.tooldefs.test.ts    # Tool schema validation: required fields,
│                             #   no duplicate names, property integrity (6 tests)
└── api.test.ts               # Express endpoint integration tests: health,
                              #   agent chat, filesystem APIs, project detection,
                              #   system stats (12 tests)
```

Test layers

Layer	What's tested	Mock strategy
Tool definitions	Every tool has valid JSON Schema, no duplicate names, required params have matching properties	None (static validation)
Filesystem tools	`runFsTool()` for each filesystem tool with real temp directories	Real filesystem
Command tools	`run_command` executes, blocks servers, returns exit codes; `read_command_output` pagination and filtering	Real `spawn`
Agent loop	`agentLoop()` and `agentLoopStream()`: tool selection logic, multi-turn loops, browser/permission handoff, iteration limits, reasoning content passthrough	Mocked DeepSeek API
API endpoints	Express routes: `/api/chat/agent`, `/api/chat/agent/stream`, `/api/health`, `/api/system/stats`, `/api/project/detect`, filesystem CRUD, session cleanup	Mocked DeepSeek, real supertest

Writing new tests

1. Tests use Vitest globals (`describe`, `it`, `expect`, `vi`, `beforeEach`, `afterEach`)
2. For agent loop tests, mock `chatDeepSeekTool` or `chatDeepSeekToolStream` from `server/deepseek.ts` to return controlled responses
3. Filesystem/command tests use `runFsTool()` with real temp directories created via `fs.mkdtempSync()`
4. API tests use `supertest` against the exported `app` from `server/index.ts`
5. Clean up temp directories in `afterEach` hooks

Token Optimization (ITR + Context Caching + Live Compaction)

Harness uses four layers to reduce token usage and API costs when talking to DeepSeek:

1. Instruction-Tool Retrieval (ITR)

Instead of sending the entire system prompt on every agent turn, the prompt is broken into **14 themed chunks** in `server/agent.ts`, each with a set of trigger keywords. At each turn, `buildSystemPrompt()` selects only the chunks relevant to the current conversation context.

Chunk registry

Each chunk is a constant string paired with an array of trigger keywords:

Chunk (id)	Size	Trigger keywords (partial)	Decision
`CORE_RULES`	~700 words	—	Always included
`browser`	~400 words	`browser_`, `DOM`, `navigate`, `click`, `type`, `form`, `modal`, `dialog`, `dropdown`, `autocomplete`, `hover`	Score ≥ 2, or auto-included if any `browser_*` tool has been called
`build_fix`	~120 words	`build`, `compile`, `error`, `fix`, `edit_file`, `write_file`, `read_problems`, `run_command`, `syntax`	Score ≥ 2
`lang_js`	~200 words	`.ts`, `.tsx`, `.js`, `.jsx`, `package.json`, `typescript`, `node`, `npm`, `react`, `vite`, `import`	Score ≥ 2
`lang_python`	~250 words	`.py`, `python`, `pip`, `flask`, `django`, `traceback`, `ModuleNotFoundError`, `uvicorn`	Score ≥ 2
`lang_go`	~100 words	`.go`, `go.mod`, `go build`, `go vet`, `go test`, `go run`	Score ≥ 2
`lang_rust`	~100 words	`.rs`, `cargo`, `Cargo.toml`, `rustc`, `rust`, `clippy`	Score ≥ 2
`lang_java`	~80 words	`.java`, `pom.xml`, `build.gradle`, `maven`, `mvn`, `gradle`, `javac`	Score ≥ 2
`lang_c`	~80 words	`.c`, `.cpp`, `.h`, `CMakeLists.txt`, `gcc`, `g++`, `cmake`, `makefile`	Score ≥ 2
`lang_ruby`	~80 words	`.rb`, `Gemfile`, `ruby`, `rake`, `rspec`, `bundle`, `gem`, `rails`	Score ≥ 2
`lang_php`	~60	`.php`, `composer.json`, `php`,	Score ≥ 2

Chunk (id)	Size	Trigger keywords (partial)	Decision
	words	`laravel`, `symfony`, `wordpress`	
`lang_shell`	~60 words	`.sh`, `.bash`, `shellcheck`, `#!/bin/bash`, `bash `, `Makefile`	Score ≥ 2
`lang_general`	~80 words	—	Always included
`server_startup`	~350 words	`run_in_terminal`, `npm start`, `npm run dev`, `flask run`, `uvicorn`, `EADDRINUSE`, `port`, `listen`	Score ≥ 2
`diagnostics`	~200 words	`run_command`, `read_problems`, `read_command_output`, `terminal`, `sandbox`, `build`, `compile`, `test`, `lint`, `error`	Score ≥ 2

Full chunk registry with every trigger keyword lives in [`PROMPT_CHUNKS`](file:///d:/Work Projects/Harness/server/agent.ts#L1743-L1862).

Selection algorithm

At each agent turn, `buildSystemPrompt()` in [agent.ts](file:///d:/Work Projects/Harness/server/agent.ts#L1907-L1959) runs this flow:

```
1. Start with CORE_RULES (always present)

2. Build a combined text blob from:
   • All message content (user, assistant, tool results)
   • Tool call names extracted from assistant messages
   • IDE context (open files, diagnostics)

3. For each optional chunk:
   count = 0
   for each trigger keyword:
       if keyword (case-insensitive) appears in combined blob:
           count += 1
   if count >= 2 → INCLUDE chunk
   if count < 2  → SKIP chunk

4. Special rule: browser chunk gets auto-included (boost = +5)
   if any browser_* tool has been called this session,
   regardless of keyword matches

5. Append IDE context footer

6. Log selection stats to console:
   [ITR] prompt: 6142 chars (~1535 tokens) | 5 chunks selected, 9 skipped
   [ITR]   included: build_fix(s:6), lang_js(s:7), diagnostics(s:4)
   [ITR]   skipped:  browser(s:0), lang_python(s:0), lang_go(s:0), ...
```

Concrete example: TypeScript project, no browser interaction

The `combined` text blob for a typical TypeScript turn contains `.ts`, `package.json`, `typescript`, `edit_file`, `run_command`, `build`, `error`, `read_problems`, `import` , etc.

```
Chunk            Triggers matched              Score   Decision
──────────────────────────────────────────────────────────────
CORE_RULES       (always)                       —      ✓ INCLUDE
browser          none                           0      ✗ SKIP
build_fix        build, error, edit_file,       6      ✓ INCLUDE
                 fix, run_command, syntax
lang_js          .ts, package.json,             7      ✓ INCLUDE
                 typescript, import, npx,
                 tsc, react
lang_python      none                           0      ✗ SKIP
lang_go          none                           0      ✗ SKIP
lang_rust        none                           0      ✗ SKIP
lang_java        none                           0      ✗ SKIP
lang_c           none                           0      ✗ SKIP
lang_ruby        none                           0      ✗ SKIP
lang_php         none                           0      ✗ SKIP
lang_shell       none                           0      ✗ SKIP
lang_general     (always)                       —      ✓ INCLUDE
server_startup   none                           0      ✗ SKIP
diagnostics      run_command, read_problems,    4      ✓ INCLUDE
                 build, test
──────────────────────────────────────────────────────────────
Result: 5 chunks included, 9 skipped
```

~600 words sent vs ~8,000 if all 14 chunks → ~92% reduction in system prompt size.

Interaction with DeepSeek prefix caching

Because `buildSystemPrompt()` produces **the same output** when the conversation context is stable (same project, same language, same tool patterns), the system message stays identical across consecutive turns. DeepSeek's server-side KV cache then reuses the cached prefix tokens — so the system prompt costs **zero additional tokens** on cache hits beyond the first turn. ITR keeps the prompt small and stable, which makes cache hits more frequent.

2. Context Caching

DeepSeek API supports **automatic prefix caching**: when consecutive requests share an identical message prefix (the system message), the server reuses the KV cache for those tokens — reducing both cost and latency.

Harness leverages this in two ways:

- **Stable system messages**: Because `buildSystemPrompt()` produces the same output for the same context, the system message stays stable across turns where the detected project stack doesn't change. DeepSeek hits the prefix cache automatically for these consecutive calls.

- **Local heuristic tracking**: `server/deepseek.ts` computes a context ID from the system message content and logs a best-effort `HIT` / `MISS` line based on whether the current system-prompt hash matches the previous request.
- **API-backed cache usage**: Harness also reads the actual DeepSeek `usage` payload and extracts:
    - `prompt_tokens`
    - `completion_tokens`
    - `total_tokens`
    - `prompt_cache_hit_tokens`
    - `prompt_cache_miss_tokens`

For streamed tool-calling requests, `server/deepseek.ts` enables `stream_options.include_usage` so the final SSE chunk includes usage data. That produces a console log like:

```
[cache-api] stream model=deepseek-v4-flash prompt=1234 completion=456 total=1690
cache_hit_tokens=900 cache_miss_tokens=334 hit_rate=73%
```

Important distinction:

- The old `[cache] ... HIT/MISS ...` line is a **local Harness heuristic** based on prompt-hash reuse.
- The new `[cache-api] ...` line is based on **actual DeepSeek API usage fields**.

No extra cache-control API parameters are needed — DeepSeek handles prefix caching transparently on the server side.

2b. Sub-agent Prefix Caching

Each sub-agent (`delegate_task`) normally starts with a fresh messages array — just the system prompt and the task string. Since every delegation has a unique task, **turn 1 of every sub-agent is a cache miss**, even when delegating the same agent type repeatedly (e.g., multiple browser sub-agents).

To improve this, Harness stores a **shared message prefix** per agent type on the parent session. After a sub-agent completes, its task and summary are appended to the prefix. The next delegation of the same type prepends these older task/summary pairs before the new task, making the API message prefix identical across calls:

```
Before (5 browser sub-agents, 3 turns each):
  Sub-agent 1: [sys, task1]                  ← MISS
  Sub-agent 2: [sys, task2]                  ← MISS  (task2 ≠ task1)
  Sub-agent 3: [sys, task3]                  ← MISS
  → 5 misses (one per delegation start)
```

```
After (same scenario):
  Sub-agent 1: [sys, task1]                  ← MISS (first ever)
  Sub-agent 2: [sys, task1, summary1, task2] ← HIT on [sys, task1, summary1]
  Sub-agent 3: [sys, task2, summary2, task3] ← HIT on [sys, task2, summary2]
  → 1 miss only (first delegation)
```

What's stored (in `AgentState.subAgentPrefix`):

Field	Content	Why
Key	Agent type string (`"browser"`, `"code-writer"`, etc.)	Per-type isolation — browser sub-agents share with each other, not with code-search
Messages	Last 2 task/summary pairs (4 messages max)	Bounded growth; never stores file contents (`read_file`/`grep` results), so project changes don't cause stale context
Task content	Truncated to 500 chars	Keeps prefix compact
Summary content	Truncated to 1000 chars	Keeps prefix compact

Where it's applied:

- `runSubAgentStream` (agent.ts) — streaming path used by the SSE agent loop
- `runSubAgent` (agent.ts) — non-streaming path (researcher tasks)
- `resumeSubAgent` does NOT store prefix — it resumes an already-paused sub-agent, so the window is unchanged

This is a low-risk optimization: only task/summary pairs are shared, never tool results containing project file contents. The worst case is 4 stale summary lines in the prefix, which act as lightweight context hints rather than authoritative information.

3. Rolling History Compaction

Long-running agent sessions now compact older plain-text turns on the server before building the next model request.

- Only older **plain chat turns** are compacted: `user` messages and non-tool `assistant` replies.
- The most recent plain turns stay verbatim so the model still sees the latest local context.

- Older plain turns are merged into a bounded **history summary** stored in the in-memory agent session.
- Tool-call ordering is preserved: assistant tool calls, tool results, pending permission state, and deferred file-accept/reject state are kept as structured messages.

This means the live prompt no longer grows linearly with every user/assistant exchange in long sessions.

Current defaults in `server/agent.ts`:

Setting	Value	Effect
`HISTORY_COMPACTION_TRIGGER_MESSAGES`	`24`	Start compacting when the in-memory session grows beyond this many messages
`HISTORY_COMPACTION_TRIGGER_TOKENS`	`10000`	Also compact when the rough token estimate crosses this threshold
`HISTORY_PLAIN_MESSAGES_TO_KEEP`	`6`	Keep the latest plain turns verbatim
`HISTORY_SUMMARY_CHAR_BUDGET`	`2400`	Bound the rolling summary size

4. Tool Result Distillation

Some tool outputs are much larger than what the model usually needs on the next turn.

Harness now distills bulky command/build output before storing it back into the agent transcript:

- `run_command` stores a compact summary instead of the full raw output
- `run_in_terminal` stores a compact summary (key error/success lines) instead of the full terminal log — the full output is cached for `read_command_output`
- `read_problems` stores a compact build-check summary instead of the full compiler/linter dump
- The summary keeps the most important lines (errors, warnings, failures, URLs, success markers)
- The **full raw command output is still cached** in the command-output store and can be re-read later with `read_command_output`

This cuts repeated replay of large terminal/compiler logs while keeping the raw output available on demand.

5. Context Token Estimation

The agent footer shows a live estimate of context usage: `~N / M tokens (X%) · T turns`. This is calculated on the server each agent turn and sent to the client via the SSE `done` event.

Harness now also sends cumulative **DeepSeek API usage** in that same `done` payload when available:

- `requestCount`
- `promptTokens`
- `completionTokens`
- `totalTokens`
- `promptCacheHitTokens`
- `promptCacheMissTokens`

The footer keeps the ring based on the local estimated-context value, and adds cache status when the API returned cache usage. The compact label becomes:

```
X% · Tt · cYY%
```

Where `cYY%` is the cumulative cache hit rate derived from:

```
promptCacheHitTokens / (promptCacheHitTokens + promptCacheMissTokens)
```

The footer tooltip includes the fuller API totals: request count, prompt/completion/total tokens, and cache hit/miss token counts.

How it's calculated

`estimateStateTokens()` in `server/agent.ts` walks every field of every message:

```
totalChars =
  Σ messages (
    content.length          // user messages, assistant replies, tool results
    + tool_call_id.length   // UUIDs linking tool calls to results
    + name.length           // function names (e.g. "read_file", "grep")
    + reasoning_content?.length  // DeepSeek chain-of-thought (R1/reasoner models)
  )
  + historySummary?.length  // compacted older conversation turns
  + systemPromptChars       // ITR-selected prompt chunks (counted once per turn)

estimatedTokens = round(totalChars / 4)
```

Each turn, `buildOpenAiMessages()` calls `estimateStateTokens()` to check against `HISTORY_COMPACTION_TRIGGER_TOKENS` (10,000). The final estimate is also sent to the

client via the SSE `done` event for the usage ring in the footer. Separately, the server accumulates actual DeepSeek API usage across the run and attaches those totals to the same `usage` object.

Accuracy

Factor	Note
`chars / 4`	Rough heuristic. DeepSeek's byte-level BPE tokenizer varies: code/text is typically 2-3 chars/token, CJK ~1 char/token. Can be off by up to 2x.
System prompt	Counted. Built from ITR chunks (3-15K chars / 0.75-3.75K tokens).
`reasoning_content`	Counted. DeepSeek R1/reasoner models produce verbose chain-of-thought.
Console context	NOT counted. The IDE's diagnostic/terminal context is small and passed separately for ITR selection.
NOT_EXECUTED injections	NOT counted. These are injected by `buildOpenAiMessages` at API-call time and not stored in `state.messages`.
API token usage	Separate from the estimate. Comes from DeepSeek's `usage` payload and may not appear if the provider omits usage for a given response.

Context limit

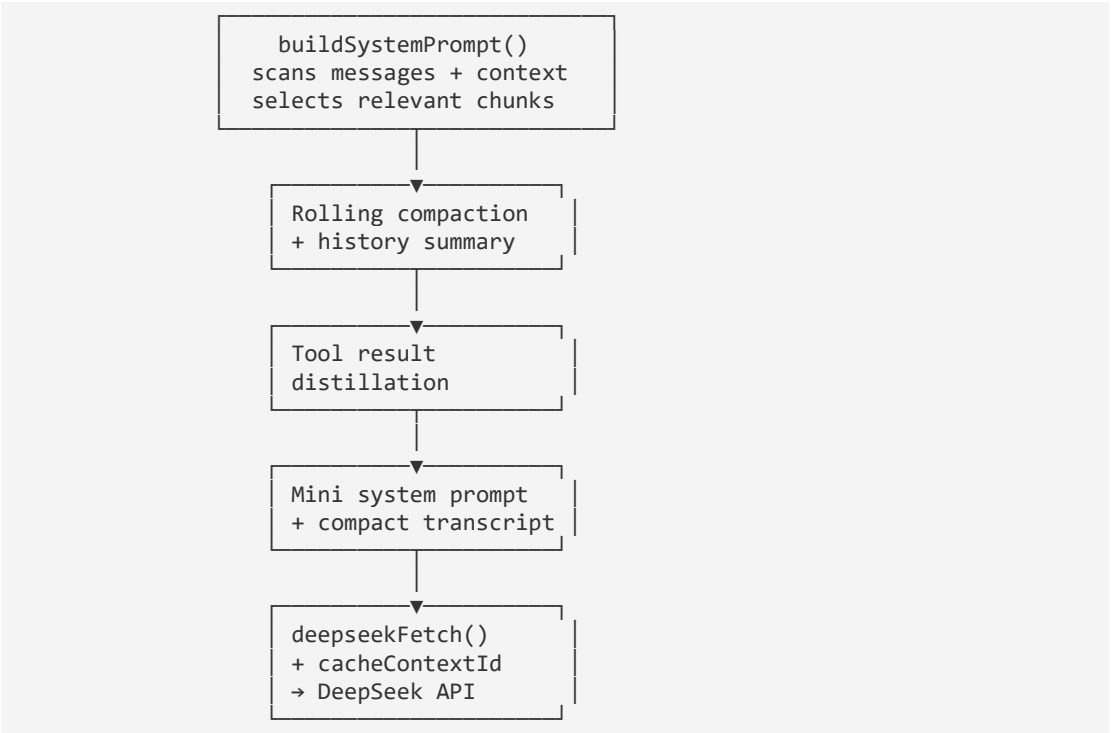
The `contextLimit` is set dynamically based on the model name:

Model pattern	Context window
`deepseek-chat` (V3), `deepseek-reasoner` (R1)	128,000 tokens
Models containing `v4`, `pro`, or `flash`	1,000,000 tokens
Unknown / custom	128,000 tokens (default)

Cumulative turns

The `turns` counter accumulates across the entire session, not per-response. The server sends `turns = iter + 1` (iterations in that agent turn), and the client sums them into `totalTurnsRef`.

Architecture



Files:

- `server/agent.ts` — `buildSystemPrompt()`, rolling history compaction, tool-result distillation, prompt assembly
- `server/deepseek.ts` — `deepseekFetch()` with `cacheContextId` parameter, cache metrics logging